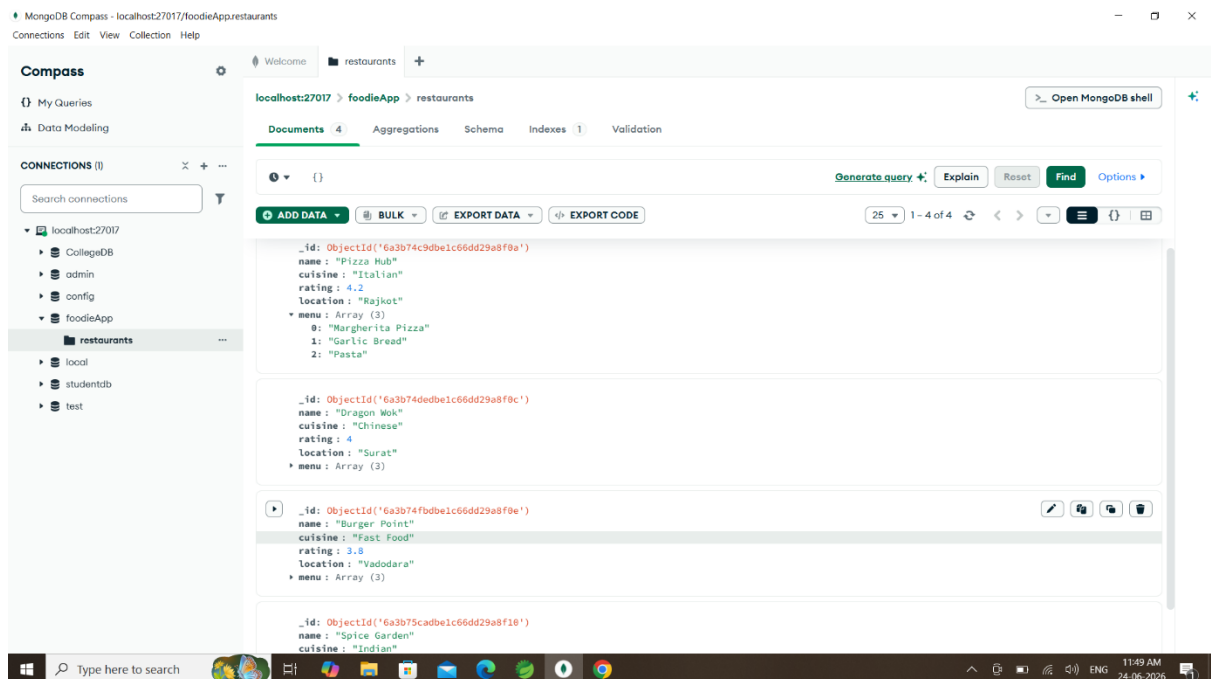


Database – NoSql – MongoDB – Assignment

SESSION 1 – NoSQL Fundamentals

1. Install MongoDB Community Edition on your system and use the mongo shell to create a new database named 'foodieApp'.
2. Inside the 'foodieApp' database, create a collection named 'restaurants' and insert a document for a Zomato-style restaurant with fields: name, cuisine, rating, and location.
3. Insert three more restaurant documents into the 'restaurants' collection, making sure to use at least one field with an array data type (for example, a 'menu' array listing popular dishes).



4. Retrieve all documents from the 'restaurants' collection and display them in the mongo shell. Identify which BSON types are used for each field.

Hint: Look for String, Number, Array, and Object types in your documents.

MongoDB Compass - localhost:27017/foodieApp.restaurants

Connections Edit View Collection Help

Compass

My Queries

Data Modeling

CONNECTIONS (1)

Search connections

localhost:27017

- CollegeDB
- admin
- config
- foodieApp
 - restaurants
- local
- studentdb
- test

localhost:27017 > foodieApp > restaurants

Documents 4 Aggregations Schema Indexes 1 Validation

Generate query Explain Reset Find Options

ADD DATA BULK EXPORT DATA EXPORT CODE

25 1 - 4 of 4

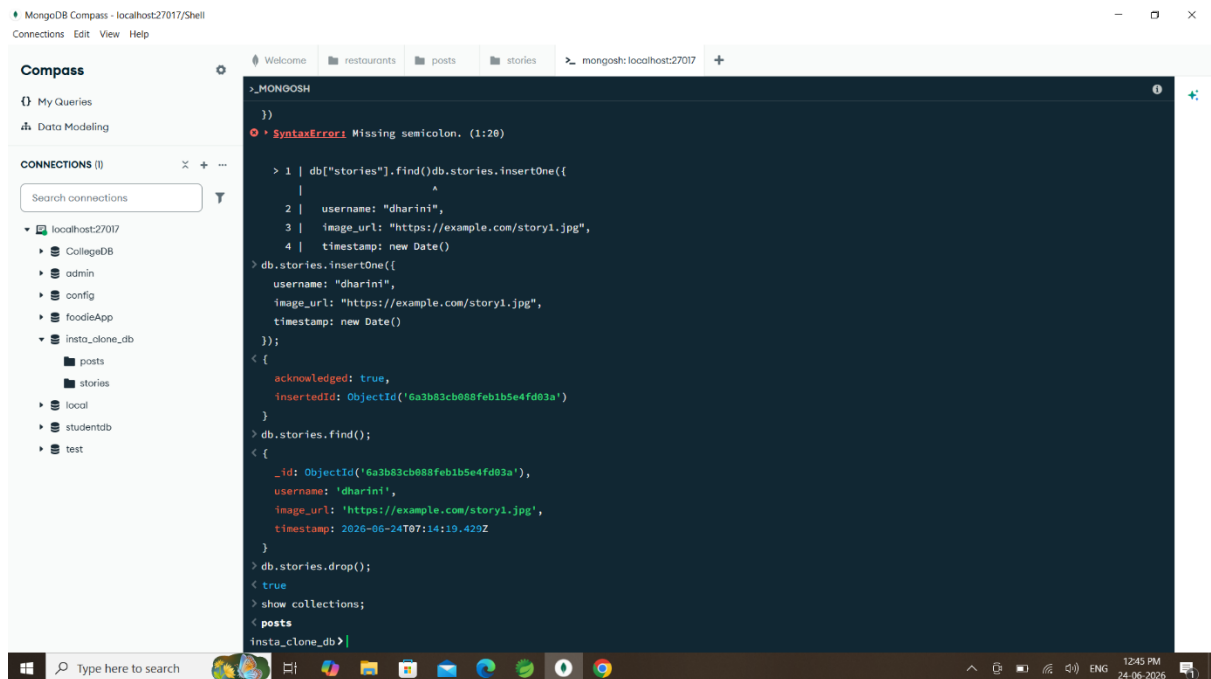
★ restaurants

	_id ObjectId	name String	cuisine String	rating Double	location String	se
1	ObjectId('6a3b74c9dbelc66...')	"Pizza Hub"	"Italian"	4.2	"Rajkot"	[]
2	ObjectId('6a3b74dedbelc66...')	"Dragon Wok"	"Chinese"	4	"Surat"	[]
3	ObjectId('6a3b74fbdbelc66...')	"Burger Point"	"Fast Food"	3.8	"Vadodara"	[]
4	ObjectId('6a3b75cadbelc66...')	"Spice Garden"	"Indian"	4.5	"Ahmedabad"	No

11:49 AM 24-06-2026

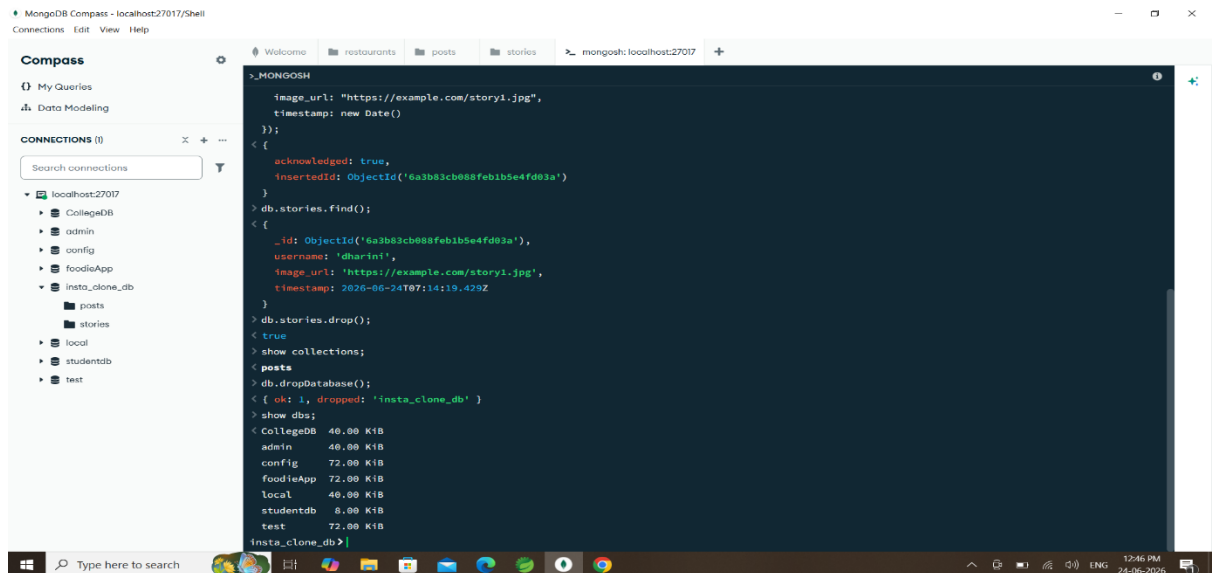
SESSION 2 – MangoDb Create / Drop Operations

1. Open MongoDB shell and create a new database named `insta_clone_db`, then switch to it using the appropriate commands.
2. Inside `insta_clone_db`, create a collection called `posts` that will store Instagram-style photo posts.
3. Create another collection named `stories` in `insta_clone_db`, and insert a sample story document with fields: `username`, `image_url`, and `timestamp`.



```
> _MONGOSH
>
> db["stories"].find()db.stories.insertOne({
  |
  | 2 | username: "dharini",
  | 3 | image_url: "https://example.com/story1.jpg",
  | 4 | timestamp: new Date()
> db.stories.insertOne({
  username: "dharini",
  image_url: "https://example.com/story1.jpg",
  timestamp: new Date()
});
< {
  acknowledged: true,
  insertedId: ObjectId('6a3b83cb088feb1b5e4fd83a')
}
> db.stories.find();
< {
  _id: ObjectId('6a3b83cb088feb1b5e4fd83a'),
  username: 'dharini',
  image_url: 'https://example.com/story1.jpg',
  timestamp: 2026-06-24T07:14:19.429Z
}
> db.stories.drop();
< true
> show collections;
< posts
insta_clone_db>
```

4. Drop the stories collection from insta_clone_db and verify that it no longer exists.
Hint: Use the drop() method on the collection and show the result with show collections.
5. Drop the entire insta_clone_db database from MongoDB, then use the show dbs command to confirm it has been deleted.

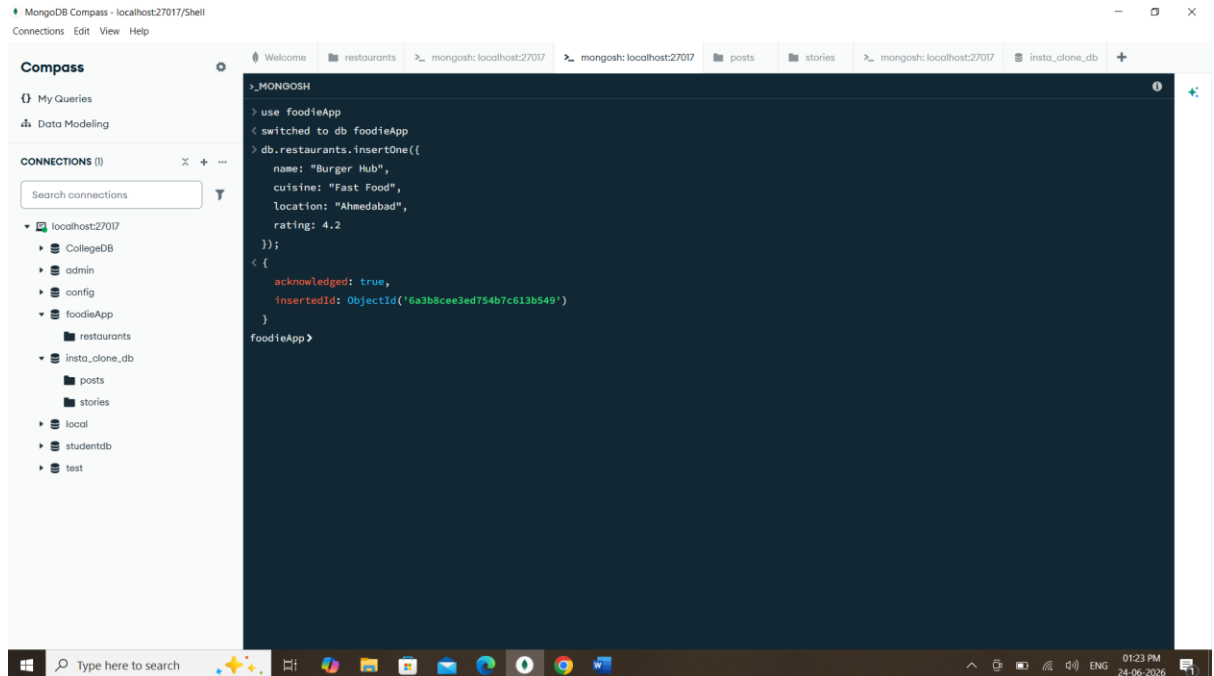


The screenshot shows the MongoDB Compass interface with the MongoDB Shell open. The left sidebar displays the database structure, including the 'insta_clone_db' database with collections 'posts' and 'stories'. The main window shows the following commands and their output:

```
> _MONGOSH
image_url: "https://example.com/story1.jpg",
timestamp: new Date()
});
< {
  acknowledged: true,
  insertedId: ObjectId('6a3b83cb088feb1b5e4fd03a')
}
> db.stories.find();
< [
  {
    _id: ObjectId('6a3b83cb088feb1b5e4fd03a'),
    username: 'dharini',
    image_url: 'https://example.com/story1.jpg',
    timestamp: 2026-06-24T07:14:19.429Z
  }
]
> db.stories.drop();
< true
> show collections;
< posts
> db.dropDatabase();
< { ok: 1, dropped: 'insta_clone_db' }
> show dbs;
< CollegeDB 46.00 KiB
admin 46.00 KiB
config 72.00 KiB
foodieApp 72.00 KiB
local 46.00 KiB
studentdb 8.00 KiB
test 72.00 KiB
insta_clone_db >
```

SESSION 3 – CRUD OPERATION

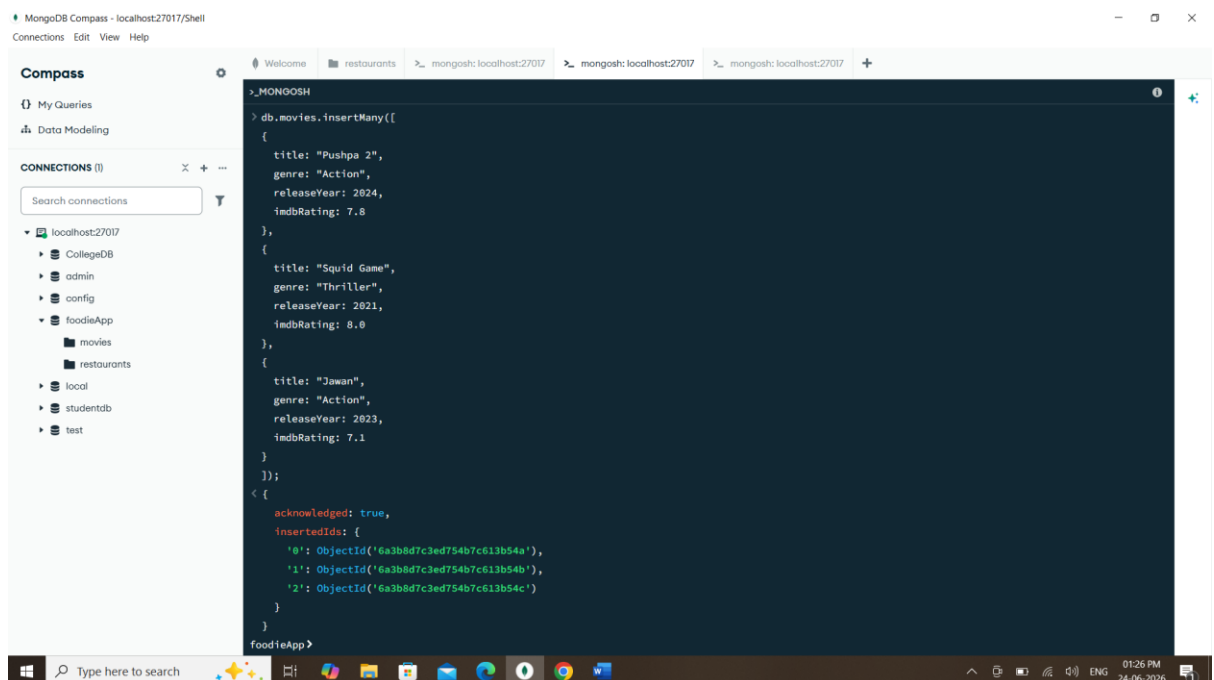
1. Create a MongoDB collection named 'restaurants' and use insertOne to add a new restaurant document with fields: name, cuisine, location, and rating.



The screenshot shows the MongoDB Compass interface. On the left, the 'CONNECTIONS' panel lists the 'localhost:27017' connection with a tree view showing databases like 'CollegeDB', 'admin', 'config', 'foodieApp', 'insta_clone_db', 'local', 'studentdb', and 'test'. The 'foodieApp' database is selected, and the 'restaurants' collection is visible. The main editor shows a MongoDB shell session with the following commands and output:

```
> use foodieApp
switched to db foodieApp
> db.restaurants.insertOne({
  name: "Burger Hub",
  cuisine: "Fast Food",
  location: "Ahmedabad",
  rating: 4.2
});
{
  acknowledged: true,
  insertedId: ObjectId('6a3b8cee3ed754b7c613b549')
}
```

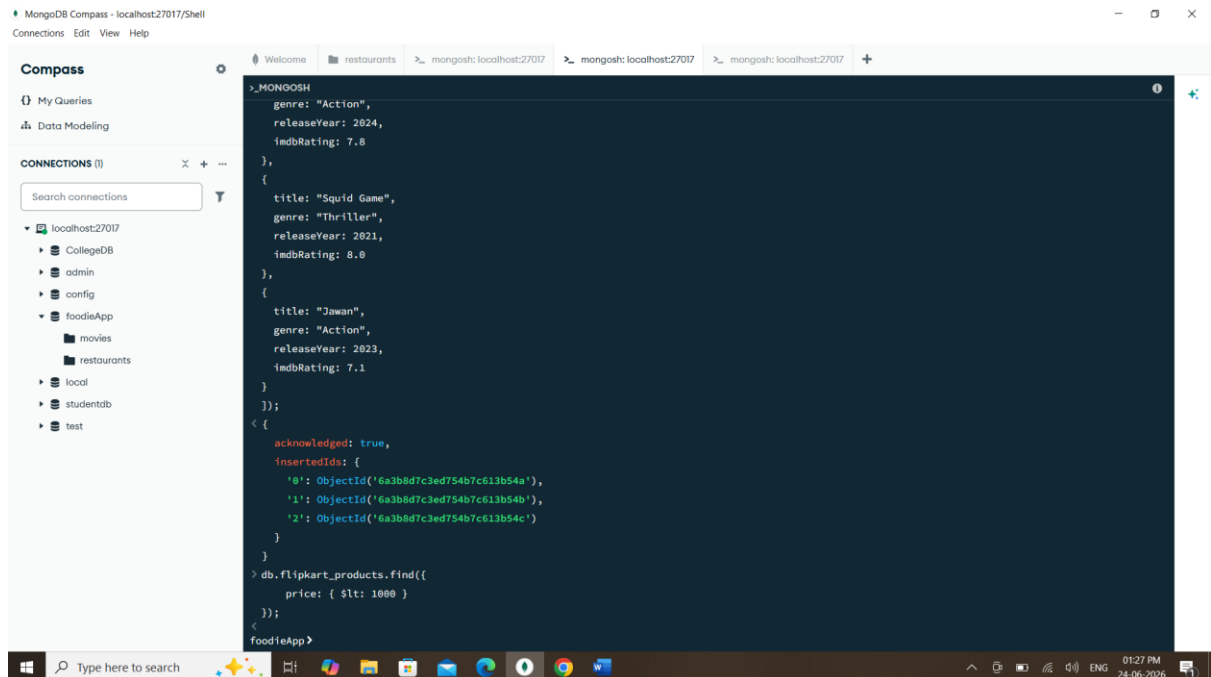
2. Use insertMany to add 3 new documents to the 'movies' collection, each representing a different movie with fields: title, genre, releaseYear, and imdbRating. Hint: Think of movies currently trending on BookMyShow or Netflix.



The screenshot shows the MongoDB Compass interface. On the left, the 'CONNECTIONS' panel lists the 'localhost:27017' connection with a tree view showing databases like 'CollegeDB', 'admin', 'config', 'foodieApp', 'insta_clone_db', 'local', 'studentdb', and 'test'. The 'foodieApp' database is selected, and the 'movies' collection is visible. The main editor shows a MongoDB shell session with the following commands and output:

```
> db.movies.insertMany([
  {
    title: "Pushpa 2",
    genre: "Action",
    releaseYear: 2024,
    imdbRating: 7.8
  },
  {
    title: "Squid Game",
    genre: "Thriller",
    releaseYear: 2021,
    imdbRating: 8.0
  },
  {
    title: "Jawan",
    genre: "Action",
    releaseYear: 2023,
    imdbRating: 7.1
  }
]);
{
  acknowledged: true,
  insertedIds: {
    '0': ObjectId('6a3b8d7c3ed754b7c613b54a'),
    '1': ObjectId('6a3b8d7c3ed754b7c613b54b'),
    '2': ObjectId('6a3b8d7c3ed754b7c613b54c')
  }
}
```

3. Write a query using find to display all products from a 'flipkart_products' collection where the price is less than 1000.



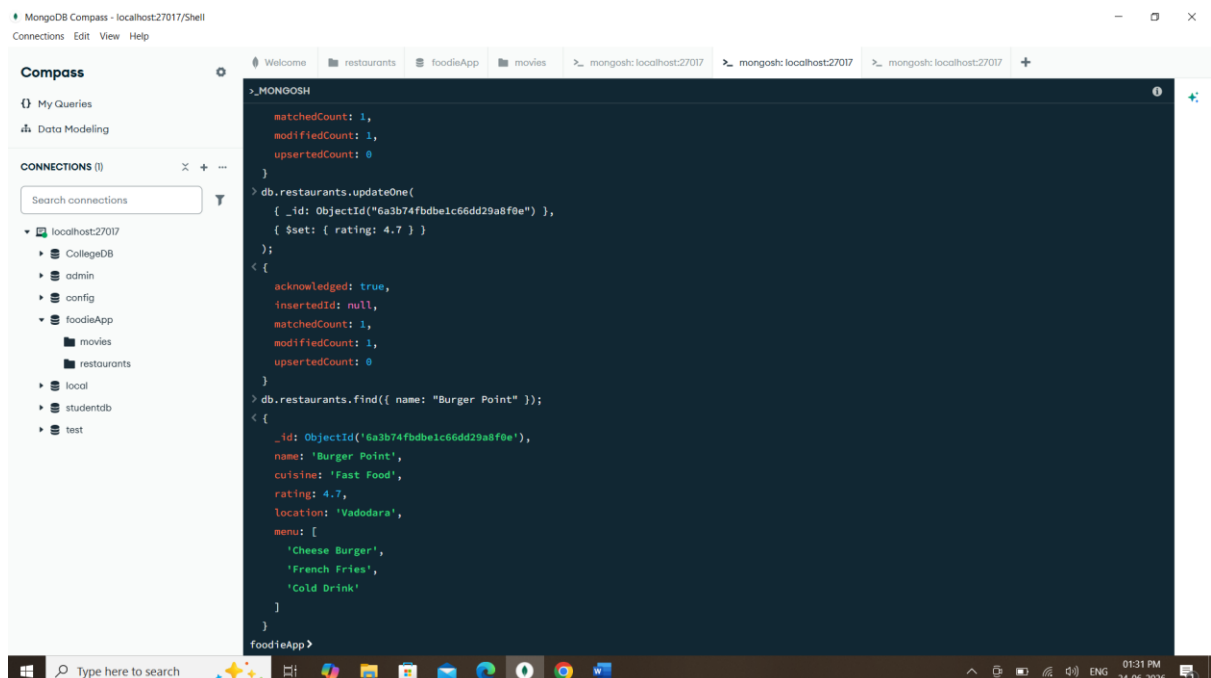
The screenshot shows the MongoDB Compass interface. The left sidebar displays the 'Connections' list with 'localhost:27017' selected, showing databases like 'CollegeDB', 'admin', 'config', 'foodieApp', 'local', 'studentdb', and 'test'. The 'foodieApp' database is expanded, showing 'movies' and 'restaurants' collections. The main editor shows a MongoDB shell session with the following code:

```
> use foodieApp
> use restaurants
> use flipkart_products
> find({
  price: { $lt: 1000 }
})
```

The output shows three documents:

```
{
  genre: "Action",
  releaseYear: 2024,
  imdbRating: 7.8
},
{
  title: "Squid Game",
  genre: "Thriller",
  releaseYear: 2021,
  imdbRating: 8.0
},
{
  title: "Jawan",
  genre: "Action",
  releaseYear: 2023,
  imdbRating: 7.1
}
```

4. Update the rating of a restaurant named 'Burger Hub' in the 'restaurants' collection to 4.7 using updateOne.



The screenshot shows the MongoDB Compass interface. The left sidebar displays the 'Connections' list with 'localhost:27017' selected, showing databases like 'CollegeDB', 'admin', 'config', 'foodieApp', 'local', 'studentdb', and 'test'. The 'foodieApp' database is expanded, showing 'movies' and 'restaurants' collections. The main editor shows a MongoDB shell session with the following code:

```
> use foodieApp
> use restaurants
> updateOne(
  { _id: ObjectId("6a3b74fbdbe1c66dd29a8f0e") },
  { $set: { rating: 4.7 } }
)
```

The output shows the result of the update operation:

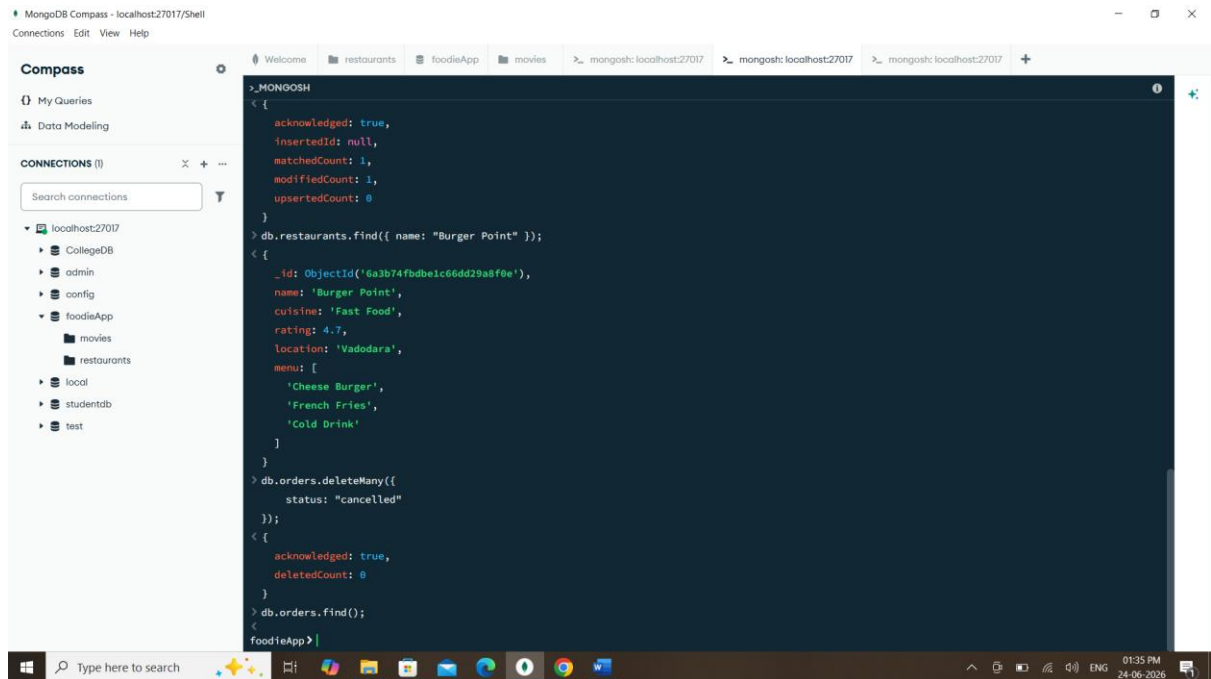
```
{
  acknowledged: true,
  insertedId: null,
  matchedCount: 1,
  modifiedCount: 1,
  upsertedCount: 0
}
```

Below the output, the updated document is shown:

```
> db.restaurants.find({ name: "Burger Point" });
{
  _id: ObjectId("6a3b74fbdbe1c66dd29a8f0e"),
  name: 'Burger Point',
  cuisine: 'Fast Food',
  rating: 4.7,
  location: 'Vadodara',
  menu: [
    'Cheese Burger',
    'French Fries',
    'Cold Drink'
  ]
}
```

5. Delete all orders from an 'orders' collection where the status is 'cancelled' using deleteMany.

Constraint: Only delete documents with status exactly equal to 'cancelled'.



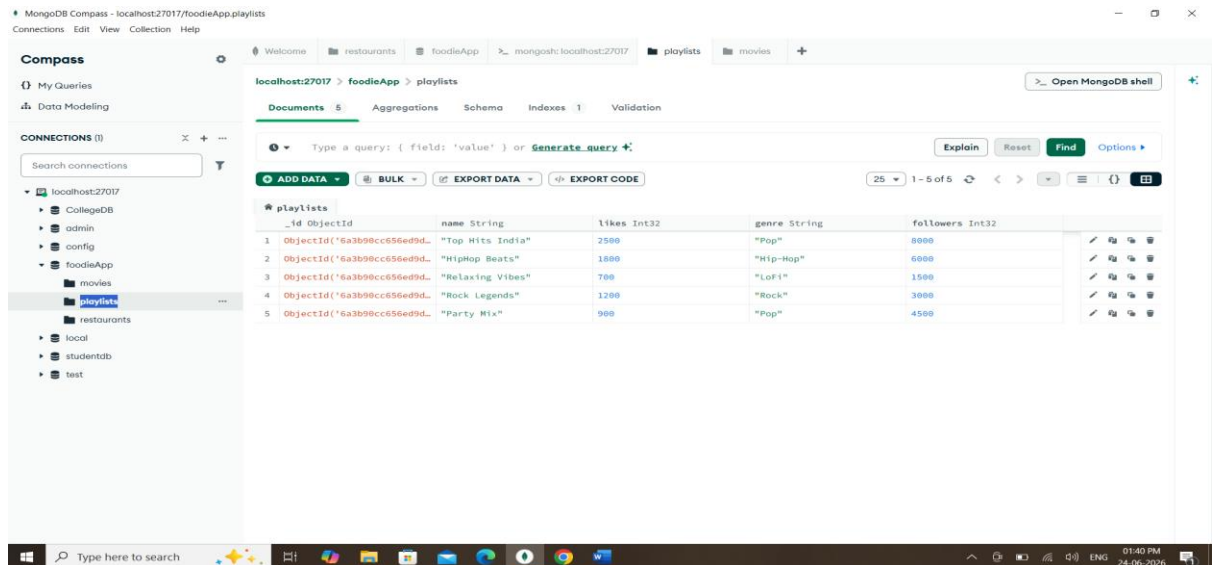
The screenshot shows the MongoDB Compass interface. The left sidebar displays the 'Connections' panel with a tree view of databases and collections. The main panel shows a MongoDB shell session with the following commands and results:

```
> MONGOSH
< {
  acknowledged: true,
  insertedCount: 0,
  matchedCount: 1,
  modifiedCount: 1,
  upsertedCount: 0
}
> db.restaurants.find({ name: "Burger Point" });
< {
  _id: ObjectId('6a3b74fbdbe1c66dd29a8f0e'),
  name: 'Burger Point',
  cuisine: 'Fast Food',
  rating: 4.7,
  location: 'Vadodara',
  menu: [
    'Cheese Burger',
    'French Fries',
    'Cold Drink'
  ]
}
> db.orders.deleteMany({
  status: "cancelled"
});
< {
  acknowledged: true,
  deletedCount: 0
}
> db.orders.find();
<
foodieApp>
```

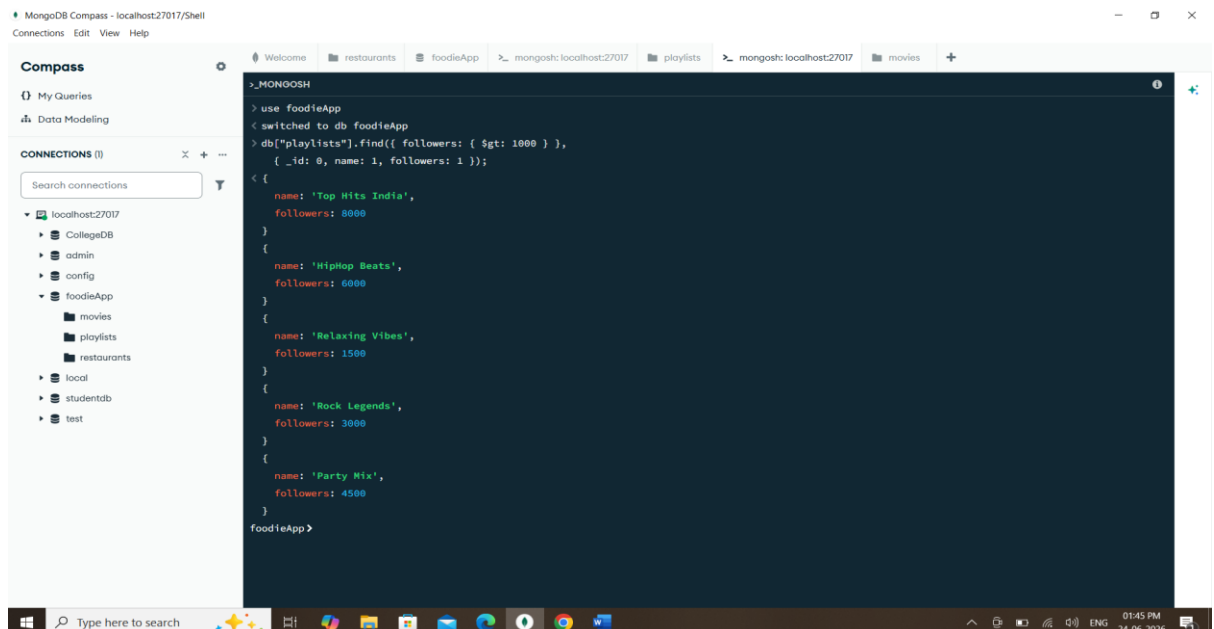
The interface includes a top menu bar with 'Connections', 'Edit', 'View', and 'Help'. The left sidebar has tabs for 'My Queries' and 'Data Modeling'. The 'Connections' panel shows a search bar and a list of connections, including 'localhost:27017' and 'localhost:27018'. The main panel shows the MongoDB shell session with the commands and results.

SESSION 4 – Query Operators

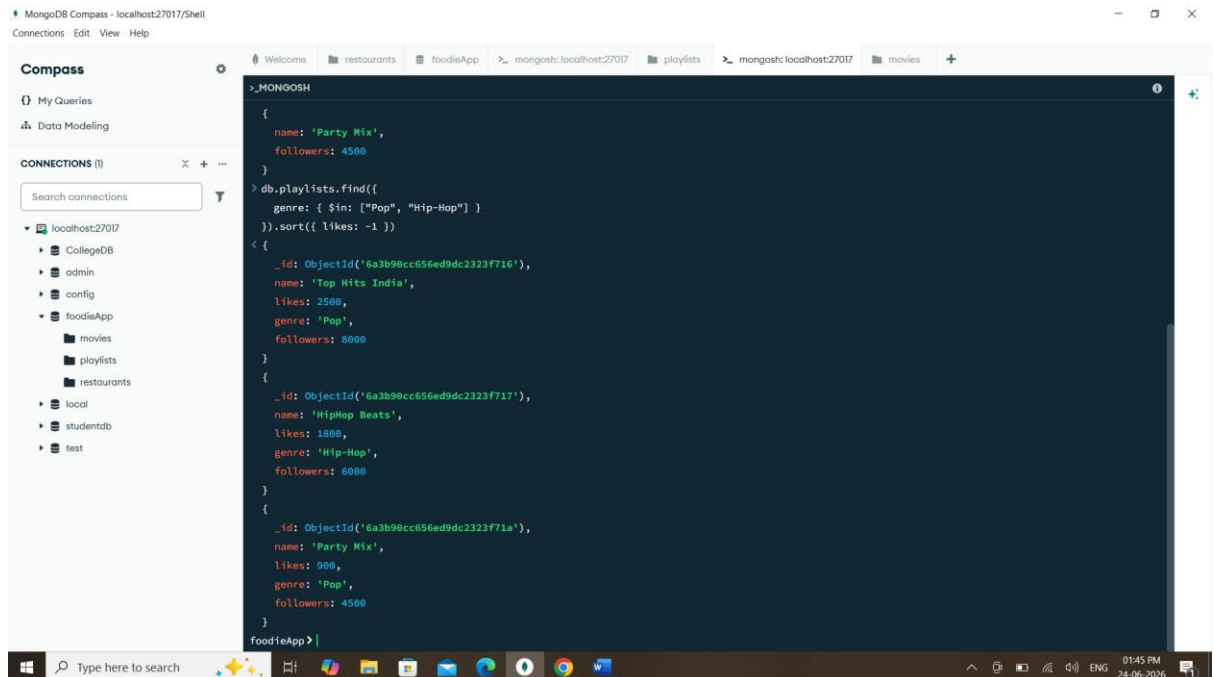
1. Insert at least 5 documents into a MongoDB collection called **playlists**, where each document represents a Spotify playlist with fields: **name** (string), **likes** (number), **genre** (string), and **followers** (number).



2. Write a MongoDB query using the **\$gt** operator to find all playlists with more than 1000 followers in the **playlists** collection and display only the **name** and **followers** fields.
Hint: Use projection to limit the fields returned.



3. Use the \$in operator to find all playlists where the genre is either 'Pop' or 'Hip-Hop' and sort the results by likes in descending order.



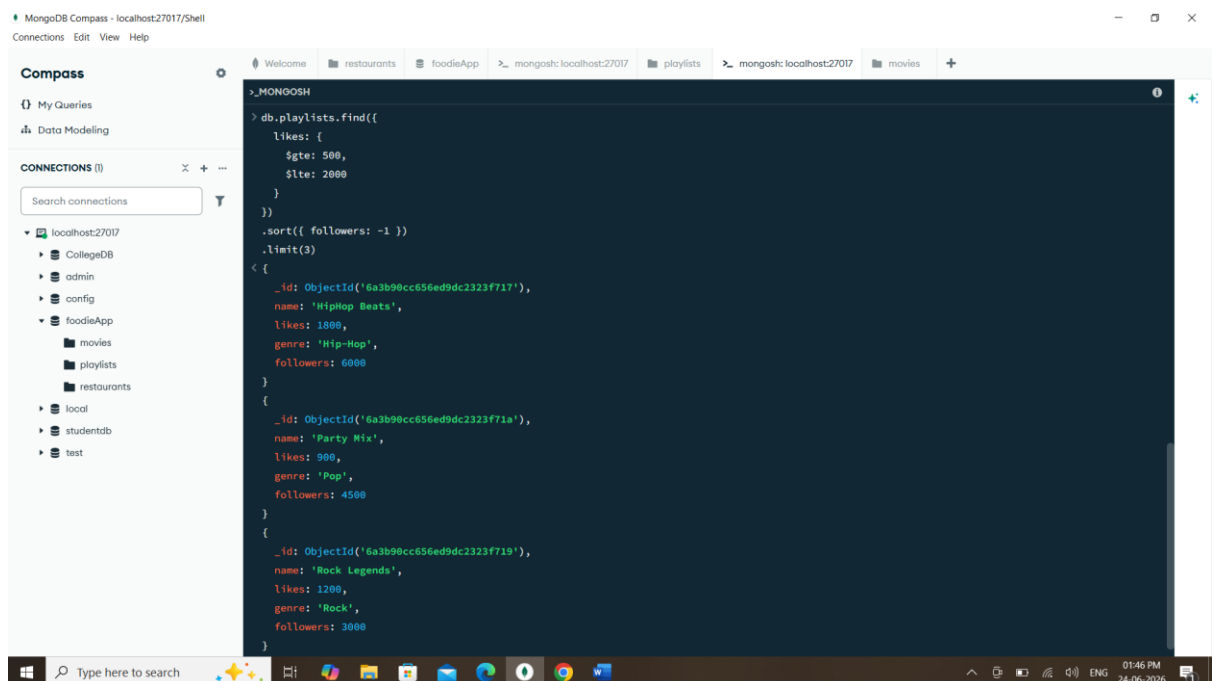
The screenshot shows the MongoDB Compass interface. The left sidebar displays the database structure with collections like 'CollegeDB', 'admin', 'config', 'foodieApp', 'movies', 'playlists', and 'restaurants'. The main editor shows a query in the 'playlists' collection:

```
> db.playlists.find(
  {
    genre: { $in: ["Pop", "Hip-Hop"] }
  },
  { sort: { likes: -1 } })
```

The results show three playlists:

- Top Hits India**: 2500 likes, Pop genre, 8000 followers.
- HipHop Beats**: 1800 likes, Hip-Hop genre, 6000 followers.
- Party Mix**: 900 likes, Pop genre, 4500 followers.

4. Query the playlists collection to find playlists with likes between 500 and 2000 (inclusive) using \$gte and \$lte, then limit the output to the top 3 playlists sorted by followers.



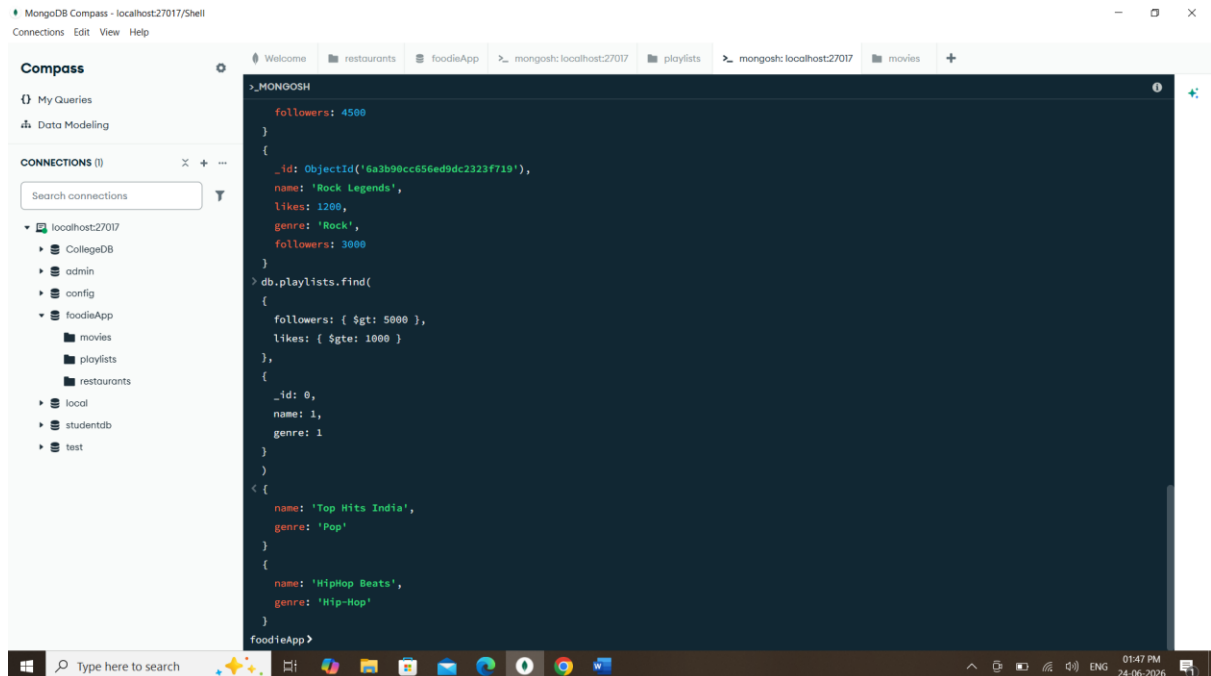
The screenshot shows the MongoDB Compass interface. The main editor shows a query in the 'playlists' collection:

```
> db.playlists.find(
  {
    likes: {
      $gte: 500,
      $lte: 2000
    }
  },
  { sort: { followers: -1 }, limit: 3 })
```

The results show three playlists:

- HipHop Beats**: 1800 likes, Hip-Hop genre, 6000 followers.
- Party Mix**: 900 likes, Pop genre, 4500 followers.
- Rock Legends**: 1200 likes, Rock genre, 3000 followers.

5. Suppose you want to recommend trending playlists like Spotify does. Write a MongoDB query that finds all playlists with more than 5000 followers and at least 1000 likes, but only show their name and genre. Explain in one line how you combined multiple query operators in your filter.

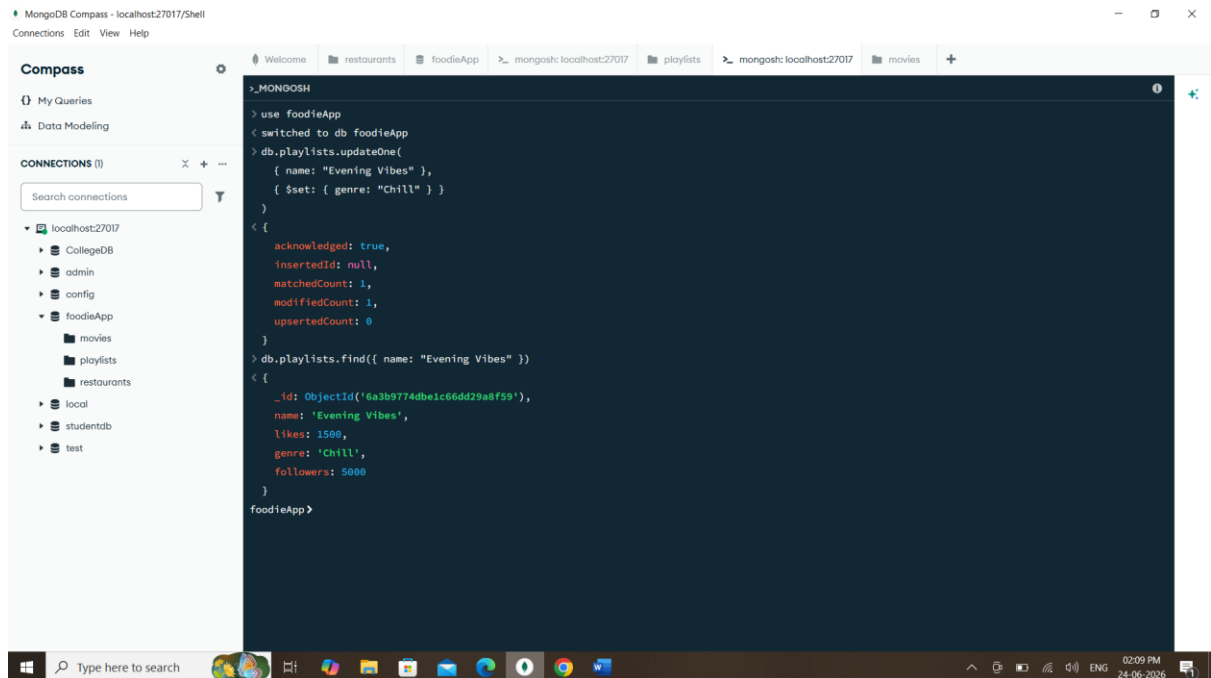


- **Explanation:**

We combined \$gt and \$gte operators in the same filter to return playlists having more than 5000 followers and at least 1000 likes.

SESSION 5 – Update Operators

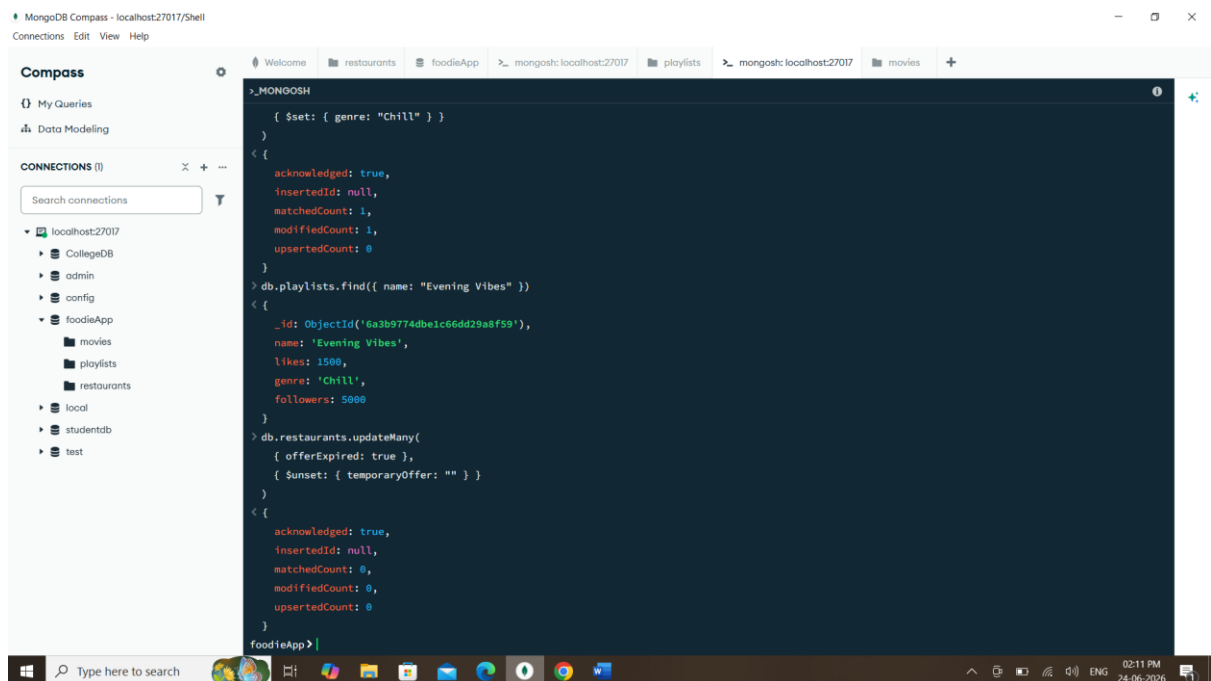
1. In a MongoDB collection named **playlists** (think Spotify), use the **\$set** operator to update the 'genre' field to 'Chill' for the playlist with name 'Evening Vibes'.



The screenshot shows the MongoDB Compass interface. The left sidebar displays the 'Connections' list with 'localhost:27017' expanded, showing databases like 'CollegeDB', 'admin', 'config', 'foodieApp', 'movies', 'playlists', 'restaurants', 'local', 'studentdb', and 'test'. The main panel shows the 'playlists' collection selected. The command window displays the following JavaScript code:

```
>_MONGOSH
> use foodieApp
< switched to db foodieApp
> db.playlists.updateOne(
  { name: "Evening Vibes" },
  { $set: { genre: "Chill" } }
)
< {
  acknowledged: true,
  insertedId: null,
  matchedCount: 1,
  modifiedCount: 1,
  upsertedCount: 0
}
> db.playlists.find({ name: "Evening Vibes" })
< {
  _id: ObjectId('6a3b9774dbe1c66dd29a8f59'),
  name: 'Evening Vibes',
  likes: 1500,
  genre: 'Chill',
  followers: 5000
}
foodieApp>
```

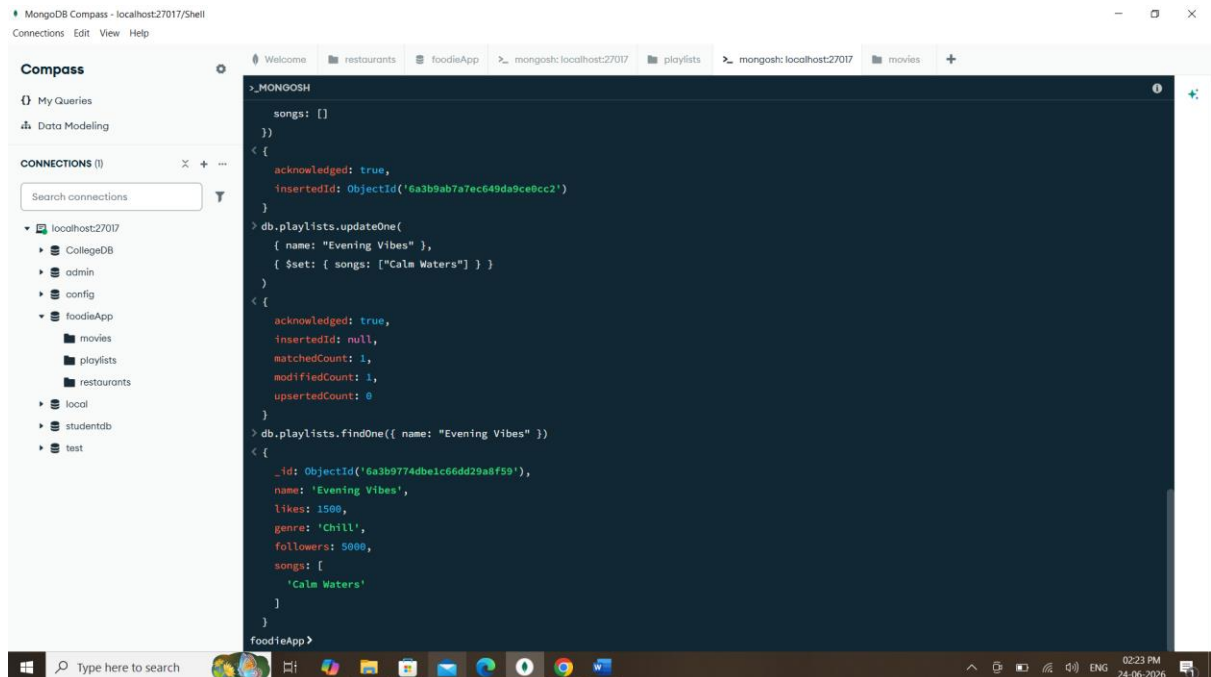
2. For a collection called **restaurants** (Zomato-style), remove the 'temporaryOffer' field from all documents where the offer has expired using the **\$unset** operator.



The screenshot shows the MongoDB Compass interface. The left sidebar displays the 'Connections' list with 'localhost:27017' expanded, showing databases like 'CollegeDB', 'admin', 'config', 'foodieApp', 'movies', 'playlists', 'restaurants', 'local', 'studentdb', and 'test'. The main panel shows the 'restaurants' collection selected. The command window displays the following JavaScript code:

```
>_MONGOSH
{ $set: { genre: "Chill" } }
}
< {
  acknowledged: true,
  insertedId: null,
  matchedCount: 1,
  modifiedCount: 1,
  upsertedCount: 0
}
> db.playlists.find({ name: "Evening Vibes" })
< {
  _id: ObjectId('6a3b9774dbe1c66dd29a8f59'),
  name: 'Evening Vibes',
  likes: 1500,
  genre: 'Chill',
  followers: 5000
}
> db.restaurants.updateMany(
  { offerExpired: true },
  { $unset: { temporaryOffer: "" } }
)
< {
  acknowledged: true,
  insertedId: null,
  matchedCount: 0,
  modifiedCount: 0,
  upsertedCount: 0
}
foodieApp>
```

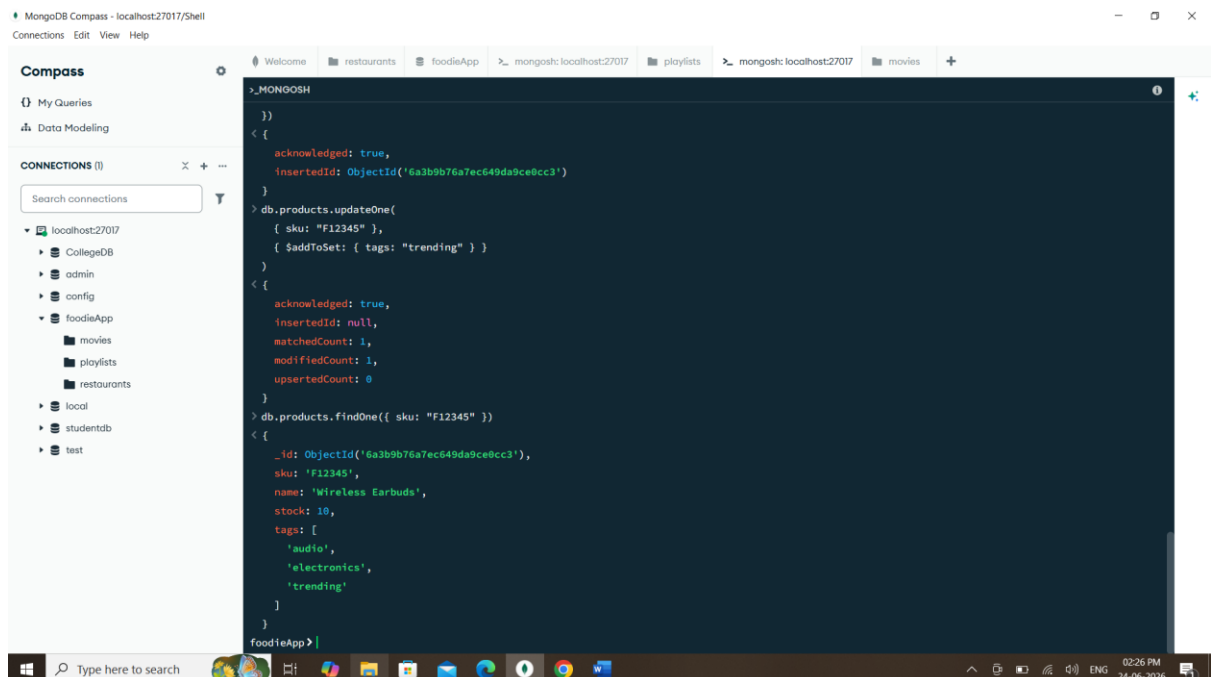
3. Using the \$push operator, add a new song 'Calm Waters' to the 'songs' array of the playlist document with _id: 101 in your playlists collection.
Hint: Use \$push to add the song to the existing array without replacing it.



The screenshot shows the MongoDB Compass interface. On the left, the 'Connections' panel shows a connection to 'localhost:27017' with a database named 'foodieApp'. The 'playlists' collection is selected. The main panel shows a shell session with the following commands and output:

```
> MONGOSH
songs: []
})
< {
  acknowledged: true,
  insertedId: ObjectId('6a3b9ab7a7ec649da9ce0cc2')
}
> db.playlists.updateOne(
  { name: "Evening Vibes" },
  { $set: { songs: ["Calm Waters"] } }
)
< {
  acknowledged: true,
  insertedId: null,
  matchedCount: 1,
  modifiedCount: 1,
  upsertedCount: 0
}
> db.playlists.findOne({ name: "Evening Vibes" })
< {
  _id: ObjectId('6a3b9774dbec66dd29a8f59'),
  name: 'Evening Vibes',
  likes: 1500,
  genre: 'Chill',
  followers: 5000,
  songs: [
    'Calm Waters'
  ]
}
```

4. In a Flipkart-like products collection, use the \$addToSet operator to ensure the tag 'trending' is added only once to the 'tags' array for the product with SKU 'F12345', even if you run the update multiple times.

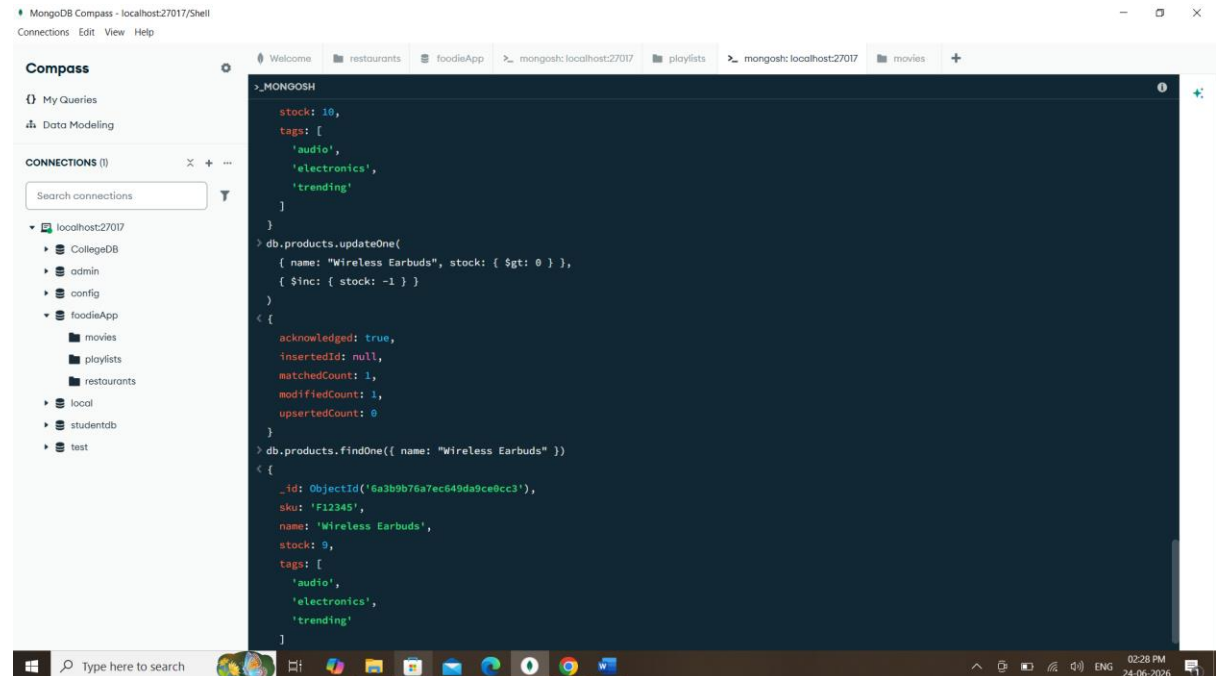


The screenshot shows the MongoDB Compass interface. On the left, the 'Connections' panel shows a connection to 'localhost:27017' with a database named 'foodieApp'. The 'products' collection is selected. The main panel shows a shell session with the following commands and output:

```
> MONGOSH
})
< {
  acknowledged: true,
  insertedId: ObjectId('6a3b9b76a7ec649da9ce0cc3')
}
> db.products.updateOne(
  { sku: "F12345" },
  { $addToSet: { tags: "trending" } }
)
< {
  acknowledged: true,
  insertedId: null,
  matchedCount: 1,
  modifiedCount: 1,
  upsertedCount: 0
}
> db.products.findOne({ sku: "F12345" })
< {
  _id: ObjectId('6a3b9b76a7ec649da9ce0cc3'),
  sku: 'F12345',
  name: 'Wireless Earbuds',
  stock: 10,
  tags: [
    'audio',
    'electronics',
    'trending'
  ]
}
```

5. Simulate a flash sale: in your products collection, use the \$inc operator to decrease the 'stock' field by 1 for the product with name 'Wireless Earbuds' every time a purchase is made.

Constraint: Do not allow stock to go below zero.

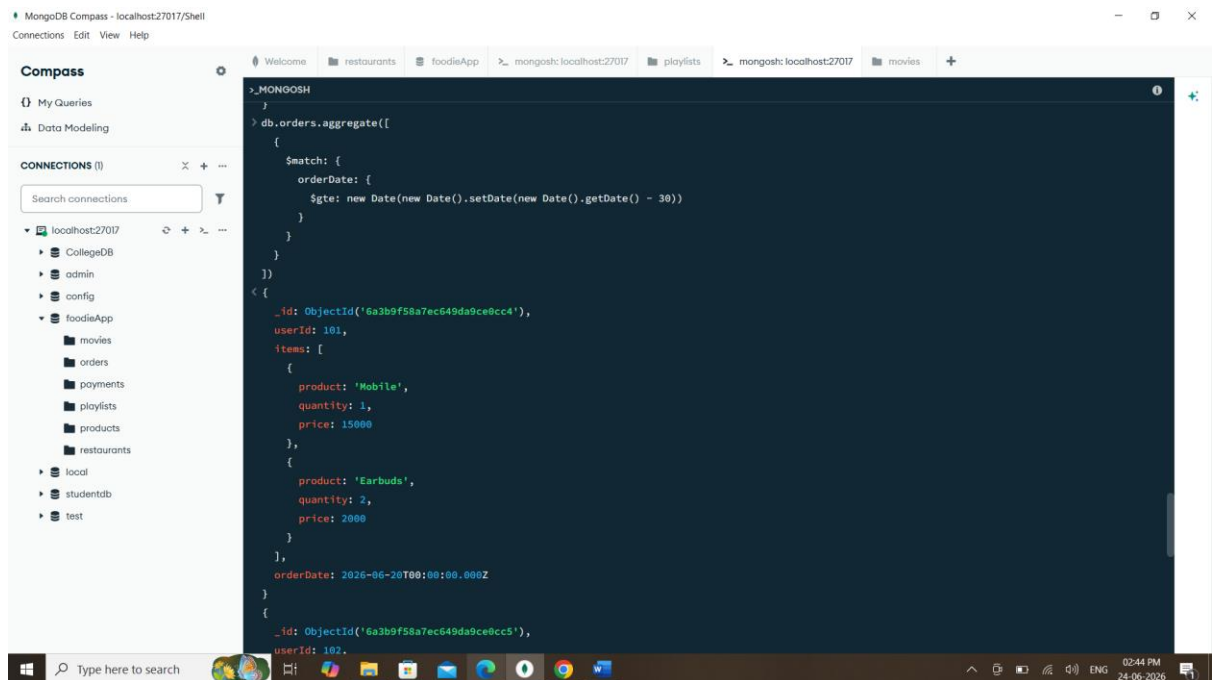


SESSION 6 – Aggregation Framework

1. Given a MongoDB collection 'orders' with documents containing fields: `userId`, `items` (array of {`product`, `quantity`, `price`}), and `orderDate`, use the `$match` stage to find all orders placed in the last 30 days.

- Query:

```
db.orders.aggregate([
  { $match: { orderDate: { $gte: new Date(new Date().setDate(new Date().getDate() - 30)) } } }
])
```

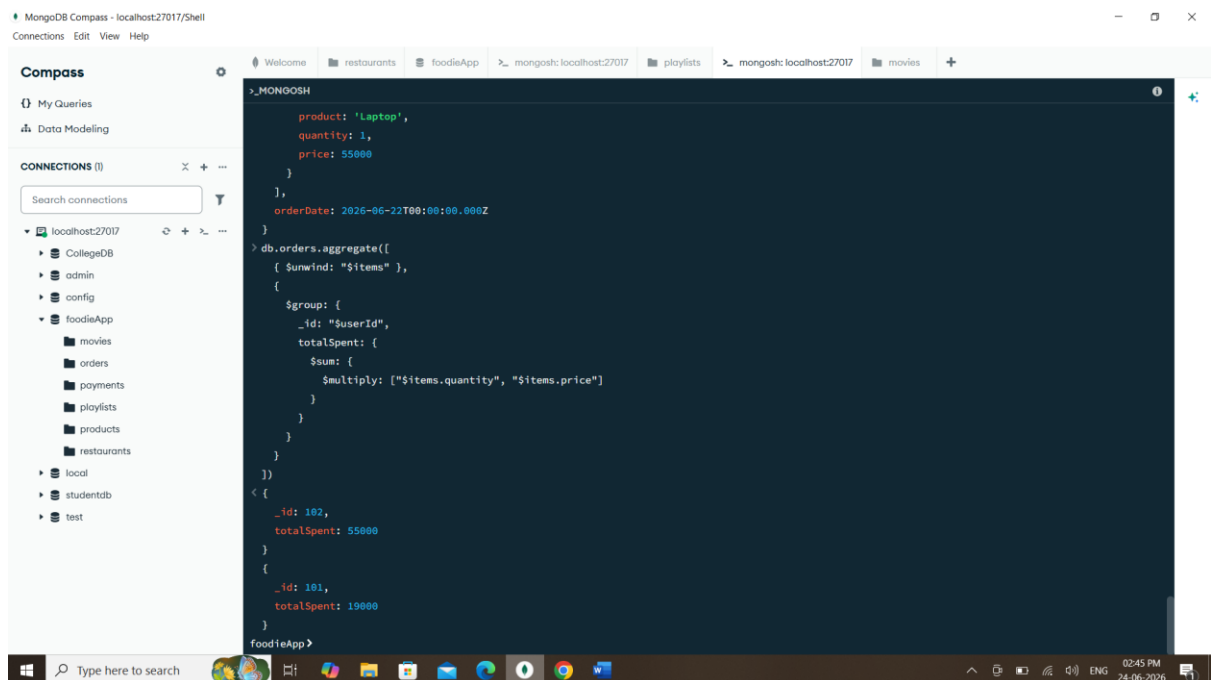


2. Using the 'orders' collection, write an aggregation pipeline that groups orders by `userId` and calculates the total amount spent by each user using the `$group` stage.

Hint: Use `$sum` to total the value of all items in each user's orders.

- Query:

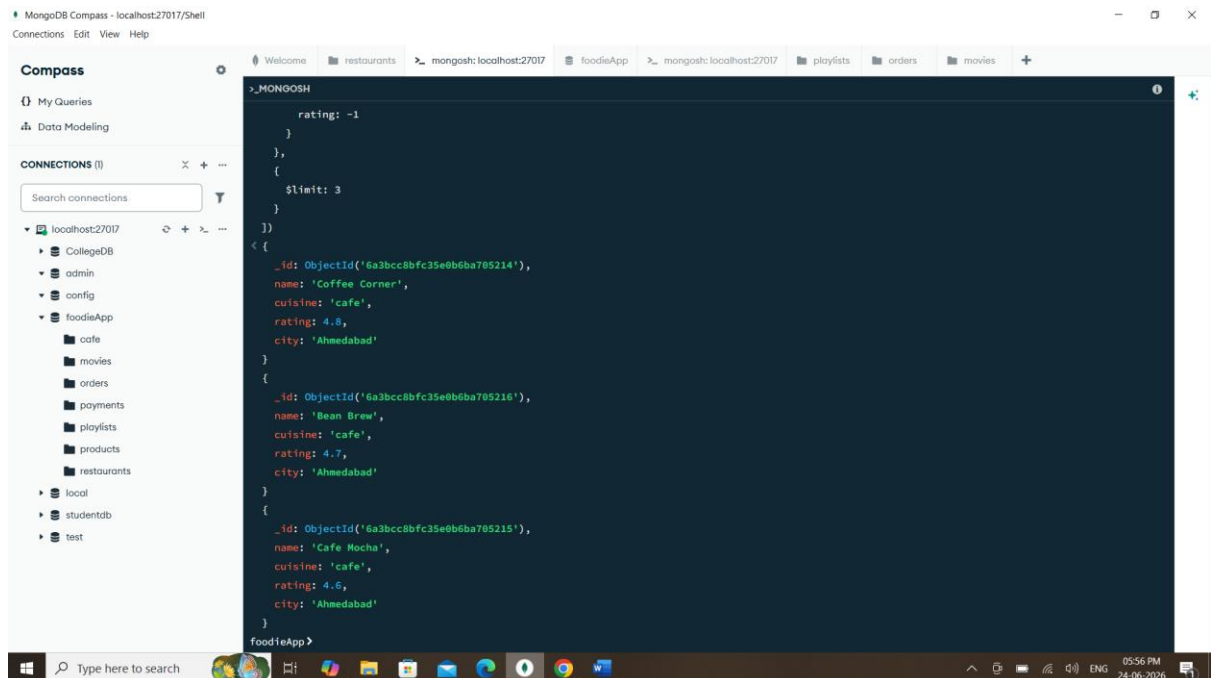
```
db.orders.aggregate([
  {$unwind:"$items"},
  {$group: {_id:"$userId",totalSpent:{$sum:{$multiply:["$items.quantity","$items.price"]}}
  }}}
])
```



3. Create an aggregation pipeline on a 'restaurants' collection (fields: name, cuisine, city, rating) to find the top 3 highest-rated cafes in Ahmedabad. Use \$match for cuisine, \$sort for rating, and \$limit for top results.

- Query:

```
db.cafe.aggregate([  
  
  {$match:{cuisine:"cafe",city:"Ahmedabad"}},  
  
  {$sort:{rating:-1}},  
  
  {$limit:3}  
  
])
```

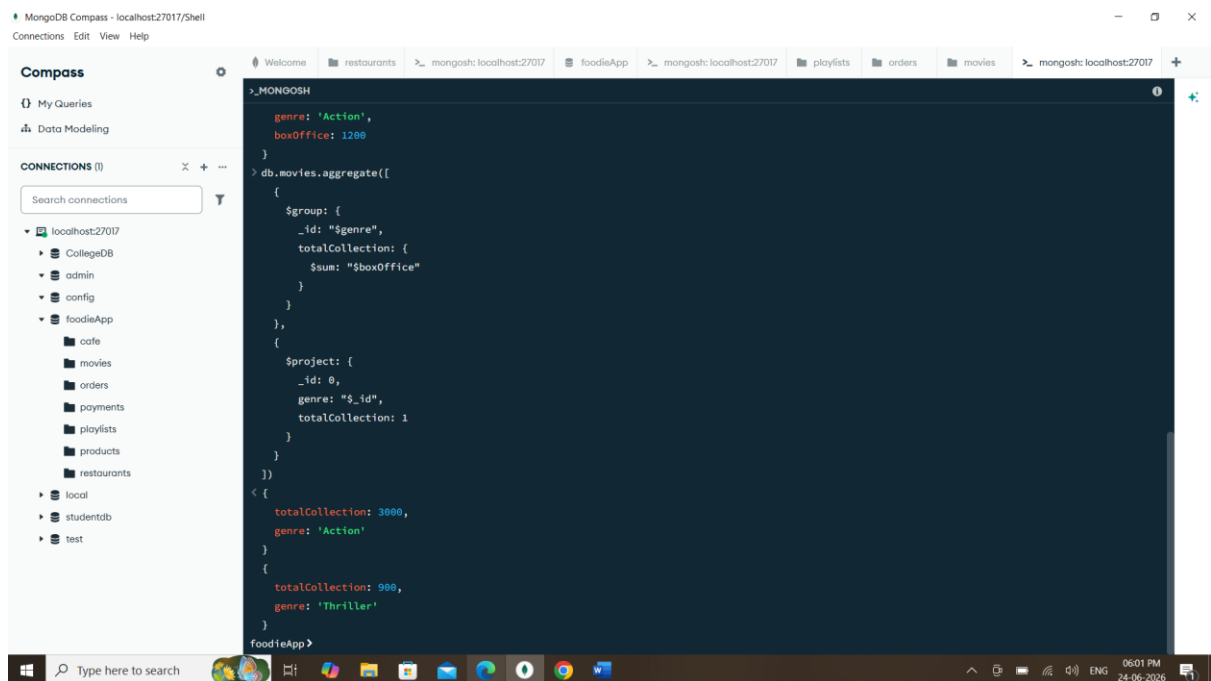


4. In a 'movies' collection (fields: title, genre, boxOffice), use \$group to calculate the total box office collection for each genre and then use \$project to display only genre and totalCollection fields in the output.

Constraint: Do not include the default _id field in your final projection.

- **Query:**

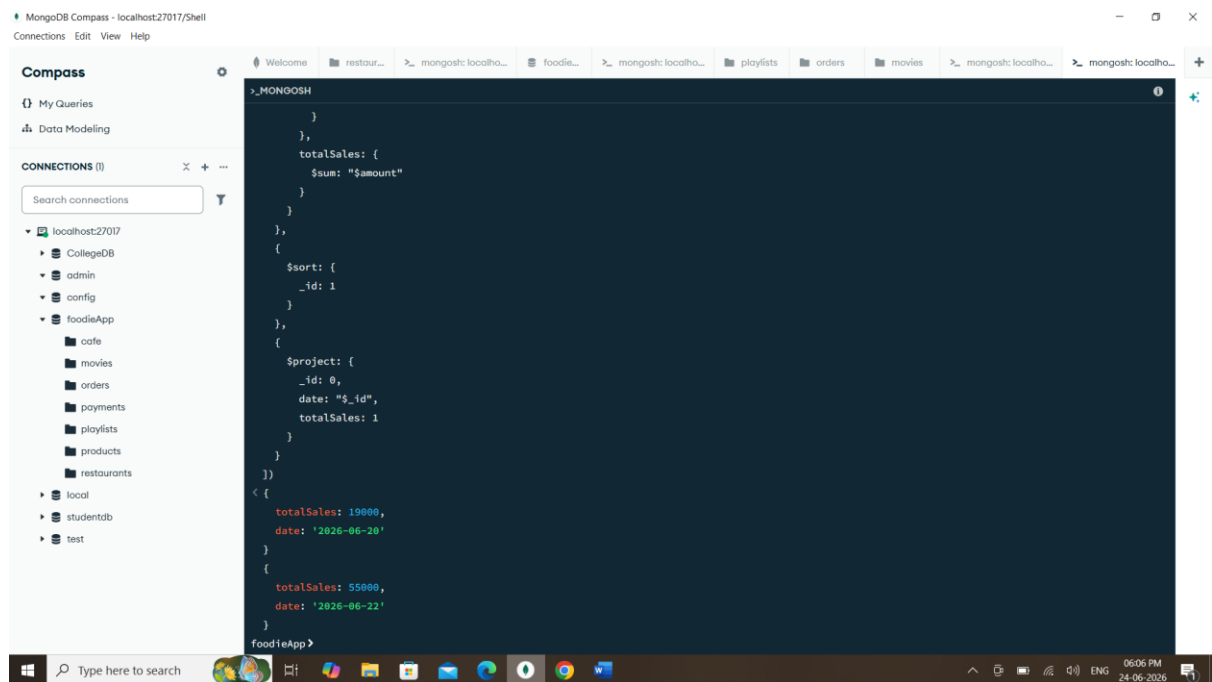
```
db.movies.aggregate([  
  
  {$group: {_id: "$genre", totalCollection: {$sum: "$boxOffice"}}},  
  
  {$project: {_id: 0, genre: "$_id", totalCollection: 1}}  
  
])
```



5. Use ChatGPT or GitHub Copilot to generate an aggregation pipeline that produces a daily sales report from a 'payments' collection (fields: userId, amount, paymentDate), showing total sales per day for the last week. Paste the generated code and briefly describe any changes you made to fit your collection's structure.

- **Query:**

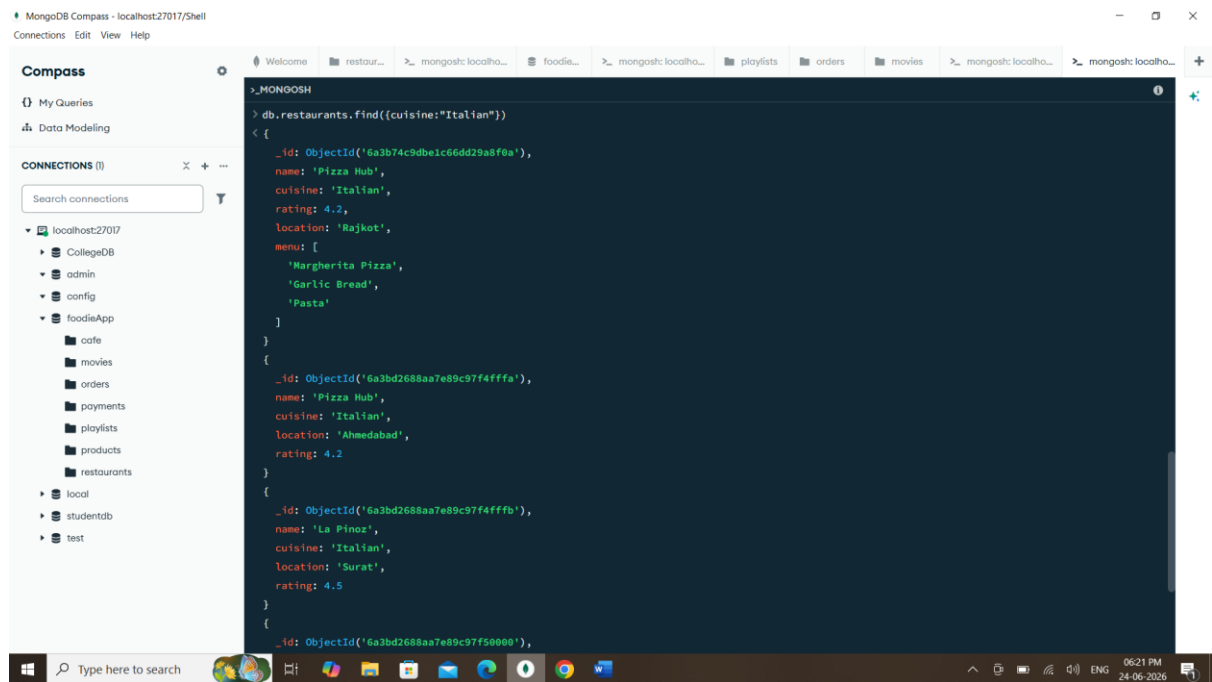
```
db.payments.aggregate([
  {$match:{$paymentDate:{$gte:new Date(new Date().setDate(new Date().getDate()-7))}}},
  {$group:{$_id:{$dateToString:{format:"%Y-%m-%d",date:"$paymentDate"}}},totalSales:{$sum:"$amount"}}},
  {$sort:{$_id:1}},
  {$project:{$_id:0,date:"$_id",totalSales:1}}
])
```



SESSION 7 – Indexing & Performance

1. Create a MongoDB collection called 'restaurants' and insert at least 10 documents with fields: name, cuisine, location, and rating. Run a find query to retrieve all restaurants with cuisine 'Italian'.

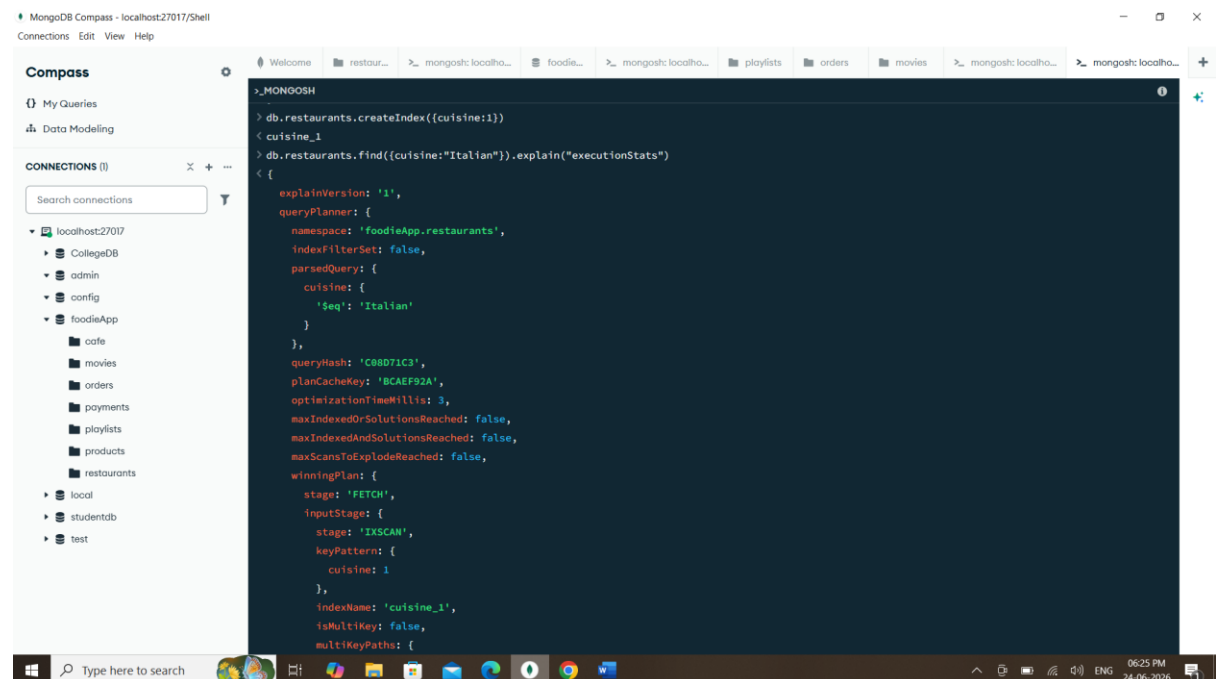
- Query:
`db.restaurants.find({cuisine:"Italian"})`



2. Add a single field index on the 'cuisine' field in the 'restaurants' collection using the `createIndex()` method. Measure and compare the query execution time for finding all 'Italian' restaurants before and after adding the index.

Hint: Use `db.restaurants.find({cuisine: 'Italian'})` and `db.restaurants.createIndex({cuisine: 1})`. Use the `.explain('executionStats')` method to check execution time.

- **Create Index:**
`db.restaurants.createIndex({cuisine:1})`
- **Check Execution Time:**
`db.restaurants.find({cuisine:"Italian"}).explain("executionStats")`
- **Observation:**
 - Before index → Collection Scan (COLLSCAN)
 - After index → Index Scan (IXSCAN)
 - Query execution time decreases.



The screenshot shows the MongoDB Compass interface. The left sidebar displays the 'Connections' list with 'localhost:27017' selected, showing a tree of databases and collections including 'restaurants'. The main panel shows the MongoDB Shell with the following commands and output:

```
> db.restaurants.createIndex({cuisine:1})
< cuisine_1

> db.restaurants.find({cuisine:"Italian"}).explain("executionStats")
< {
  explainVersion: '1',
  queryPlanner: {
    namespace: 'foodieApp.restaurants',
    indexFilterSet: false,
    parsedQuery: {
      cuisine: {
        '$eq': 'Italian'
      }
    },
    queryHash: 'C68D71C3',
    planCacheKey: 'BCAEF92A',
    optimizationTimeMillis: 3,
    maxIndexedOrSolutionsReached: false,
    maxIndexedAndSolutionsReached: false,
    maxScansToExploreReached: false,
    winningPlan: {
      stage: 'FETCH',
      inputStage: {
        stage: 'IXSCAN',
        keyPattern: {
          cuisine: 1
        },
        indexName: 'cuisine_1',
        isMultiKey: false,
        multiKeyPaths: {
```

The output indicates that the index 'cuisine_1' was successfully created and that the query for 'Italian' restaurants is now using an Index Scan (IXSCAN) instead of a Collection Scan (COLLSCAN).

3. Create a compound index on the 'cuisine' and 'location' fields in the 'restaurants' collection. Write a query to find all 'Chinese' restaurants in 'Ahmedabad' and check if the compound index is being used by the query planner.

- **Create Index:**

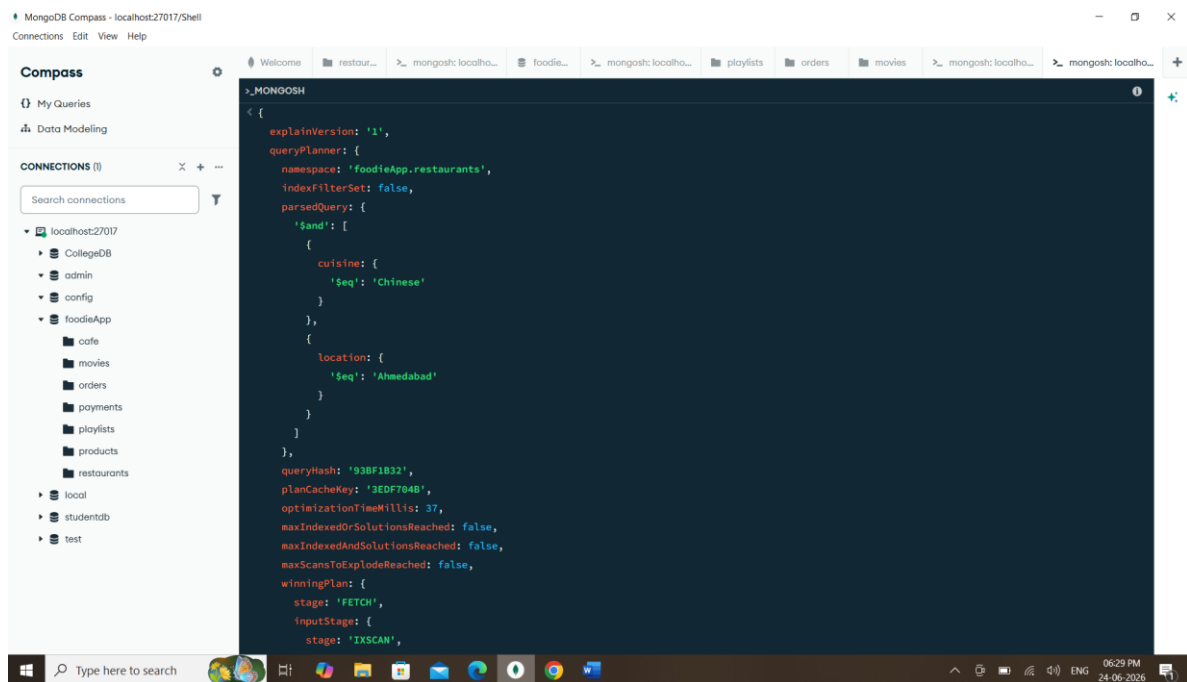
```
db.restaurants.createIndex({cuisine:1,location:1})
```

- **Query:**

```
db.restaurants.find({  
  cuisine:"Chinese",  
  location:"Ahmedabad"  
}).explain("executionStats")
```

- **Observation:**

- Query planner uses compound index.
- Faster search for cuisine + location together.



4. Drop the index you created on the 'cuisine' field and observe how the query performance changes when searching for a specific cuisine. Record your observations in 2-3 lines.

- **Drop Index:**

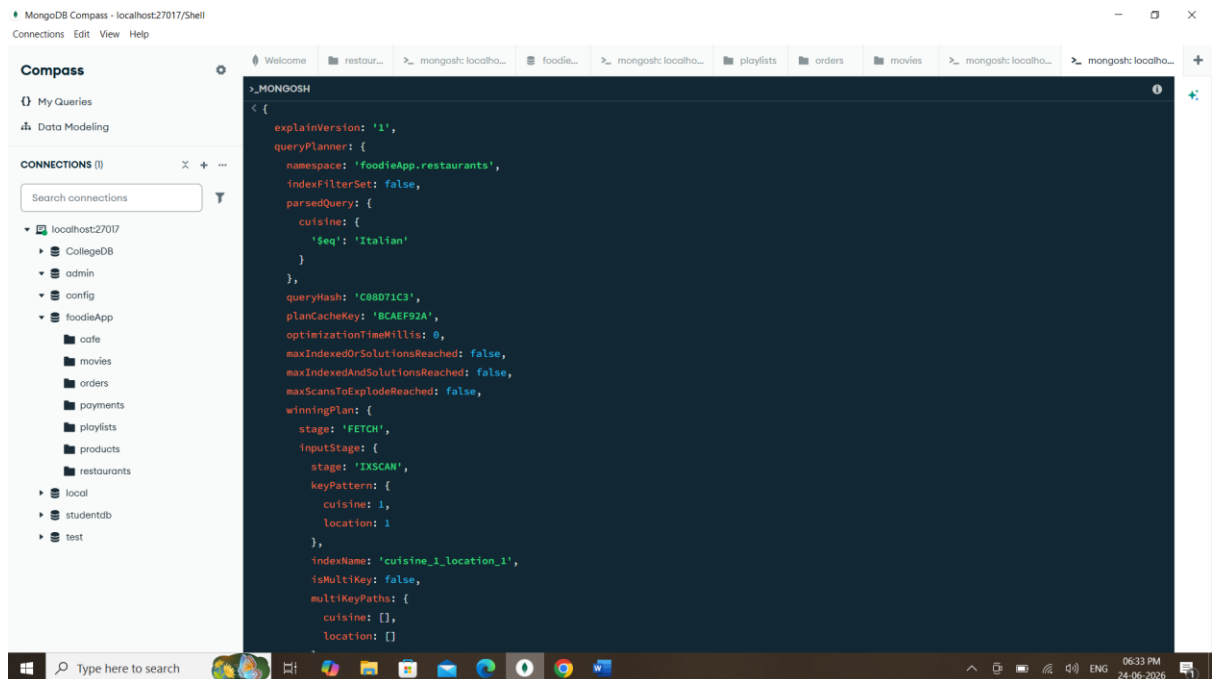
```
db.restaurants.dropIndex({cuisine:1})
```

- **Check Again:**

```
db.restaurants.find({cuisine:"Italian"}).explain("executionStats")
```

- **Observation:**

- After dropping the index, MongoDB performs a collection scan.
- Query execution time increases.
- Searching becomes slower compared to indexed queries.



5. Use ChatGPT to generate a MongoDB query that would efficiently fetch the top 5 highest rated 'Pizza' restaurants in 'Surat', making sure to use any relevant indexes. Paste the generated query and explain in one line how indexing helps in this scenario.

- **Create Relevant Index:**

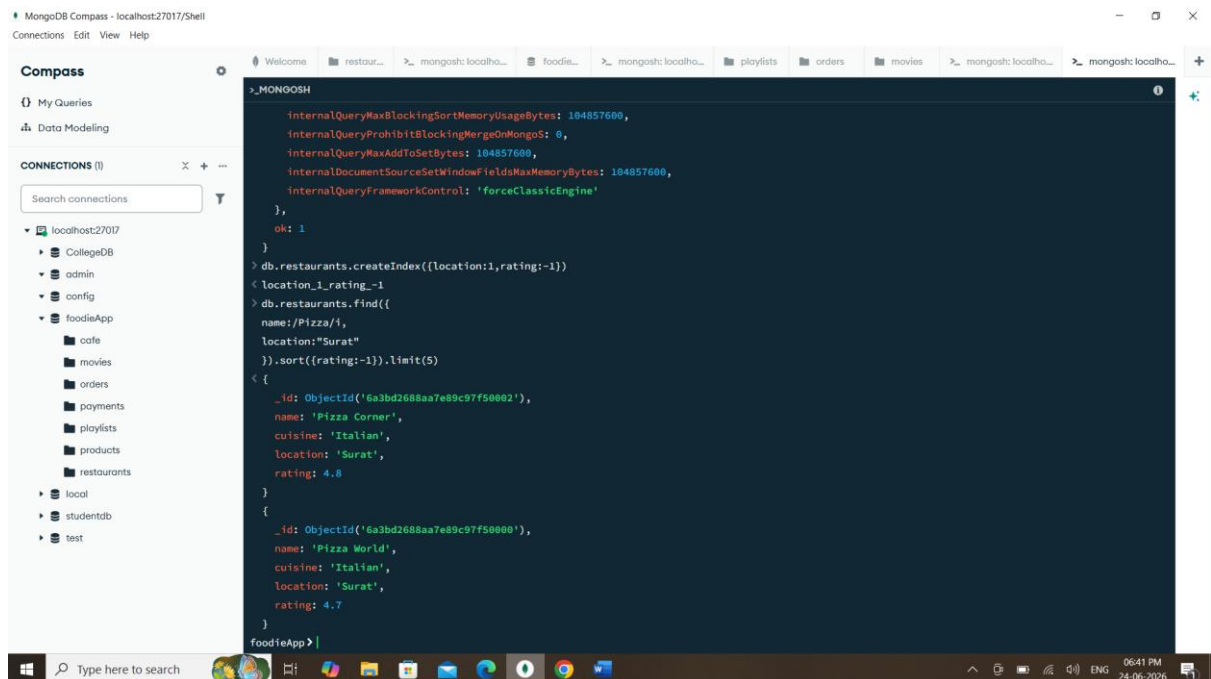
```
db.restaurants.createIndex({location:1,rating:-1})
```

- **Query:**

```
db.restaurants.find({  
  name:/Pizza/i,  
  location:"Surat"  
}).sort({rating:-1}).limit(5)
```

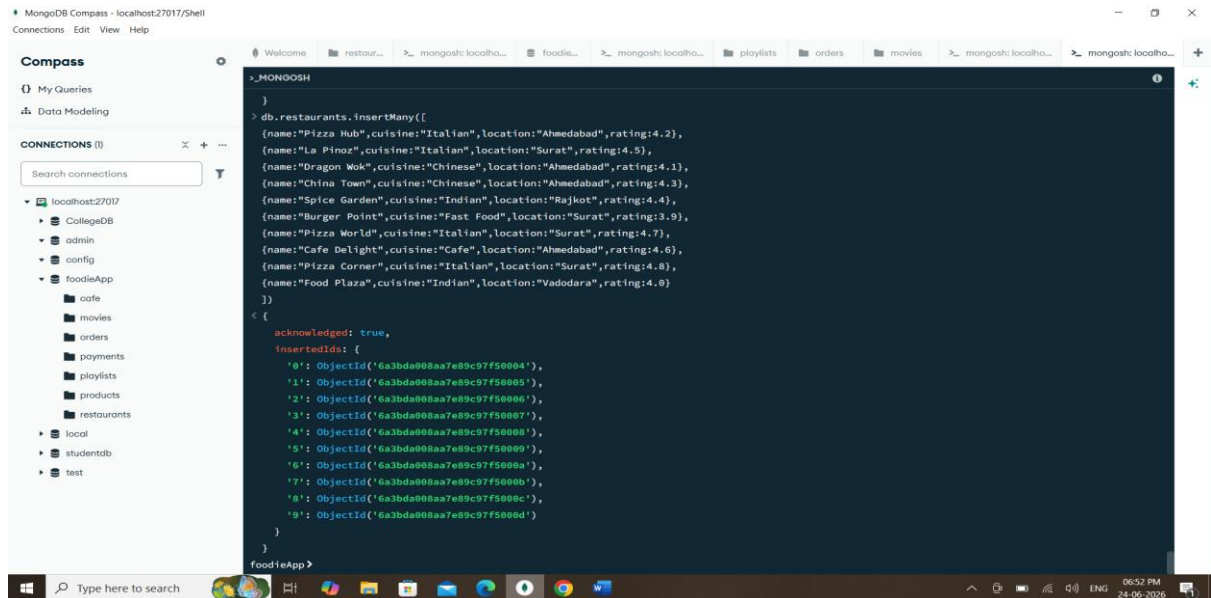
- **Explanation:**

Indexing helps MongoDB find and sort matching restaurants faster without scanning the entire collection.



SESSION 8 – Indexing & Performance

1. Create a MongoDB collection named 'restaurants' and insert 10 documents with fields: name, cuisine, location, and rating.



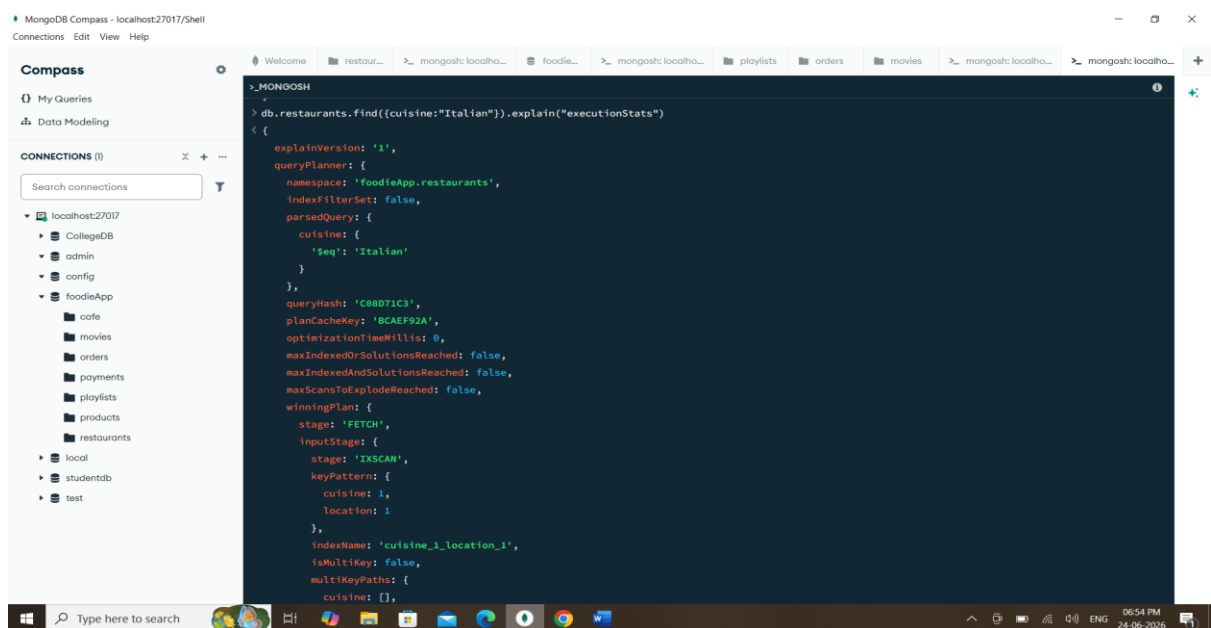
2. Query the 'restaurants' collection to find all restaurants with cuisine 'Italian' and measure the query execution time using explain('executionStats').

- Find Italian Restaurants + Execution Stats

```
db.restaurants.find({cuisine:"Italian"}).explain("executionStats")
```

- Observation:

- Before index → COLLSCAN
- Query execution time is higher.



3. Create an index on the 'cuisine' field in the 'restaurants' collection and rerun the same query as before. Compare the execution time before and after adding the index.
Hint: `Use db.restaurants.createIndex({ cuisine: 1 })` and check the 'executionTimeMillis' in the explain output.

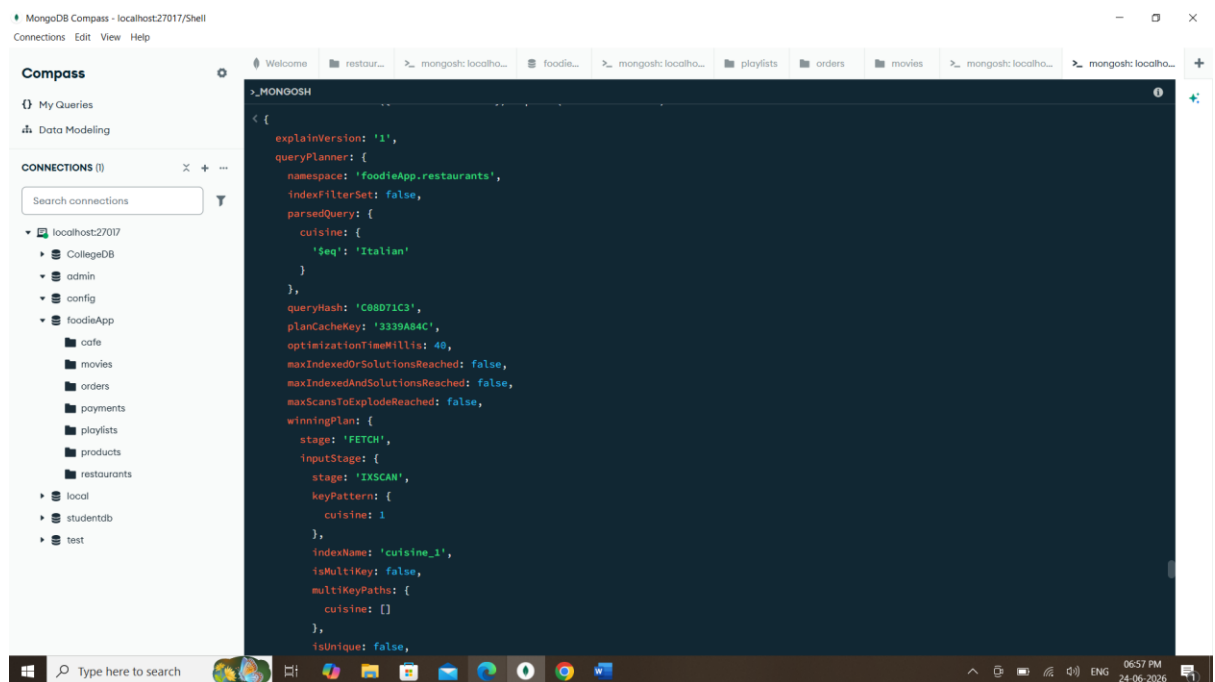
- **Create Index & Compare**

```
db.restaurants.createIndex({cuisine:1})
```

```
db.restaurants.find({cuisine:"Italian"}).explain("executionStats")
```

- **Observation:**

- After index → IXSCAN
- executionTimeMillis decreases.
- Query becomes faster.



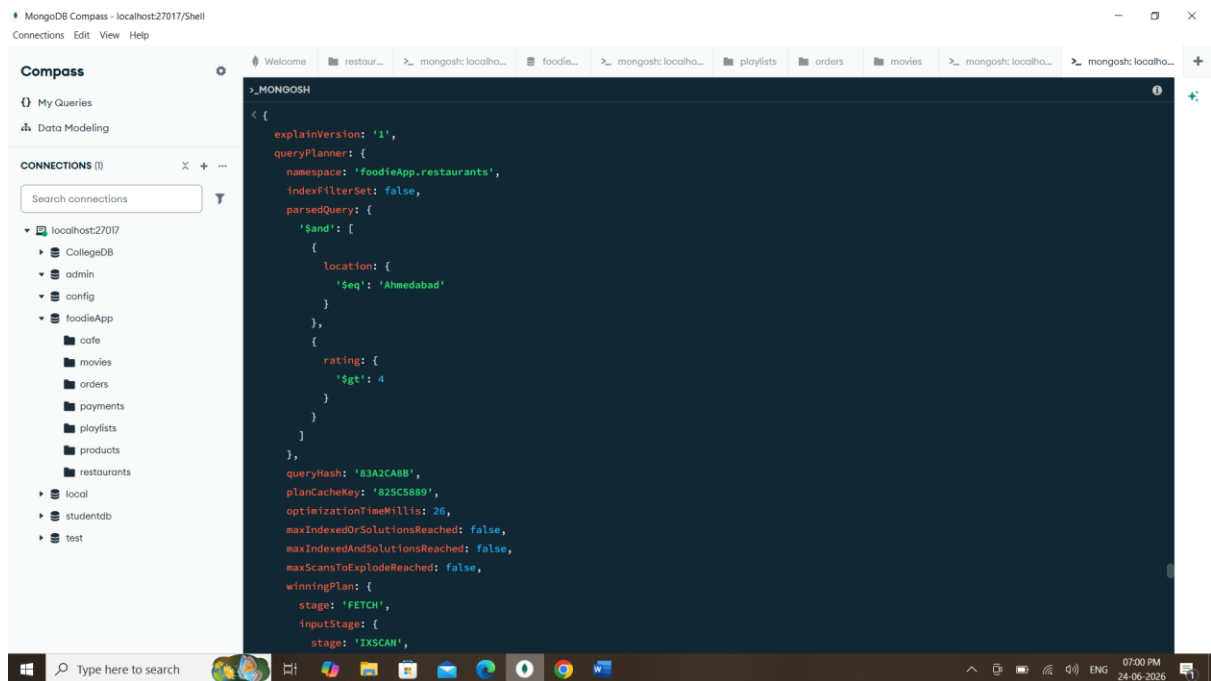
4. Add a compound index on 'location' and 'rating' in the 'restaurants' collection. Write a query to find all restaurants in 'Ahmedabad' with a rating above 4, and observe the performance impact using explain().

- **Compound Index & Query**

```
db.restaurants.createIndex({location:1,rating:1})
db.restaurants.find({
  location:"Ahmedabad",
  rating:{ $gt:4}
}).explain("executionStats")
```

- **Observation:**

- Compound index is used.
- Search performance improves.



5. Drop all indexes from the 'restaurants' collection except the default _id index and verify by listing the current indexes.

Constraint: Use `db.restaurants.dropIndexes()` and `db.restaurants.getIndexes()` to confirm.

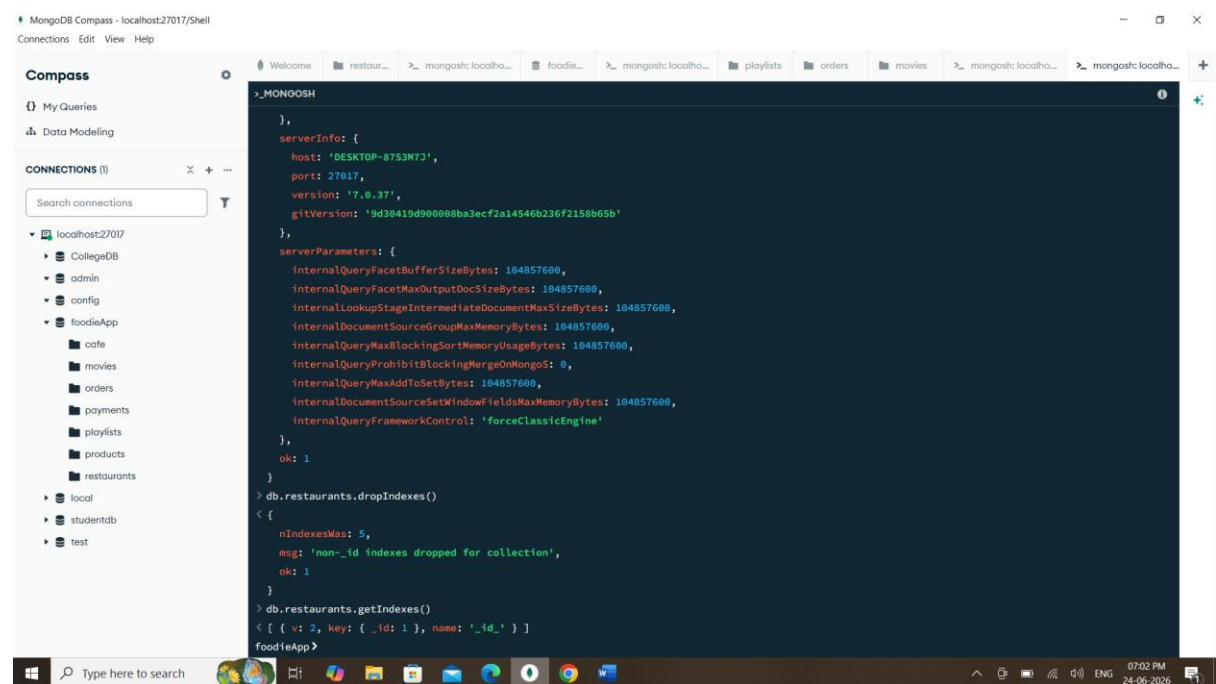
- **Drop All Indexes & Verify**

`db.restaurants.dropIndexes()`

`db.restaurants.getIndexes()`

- **Observation:**

- All custom indexes are removed.
- Only the default _id_ index remains



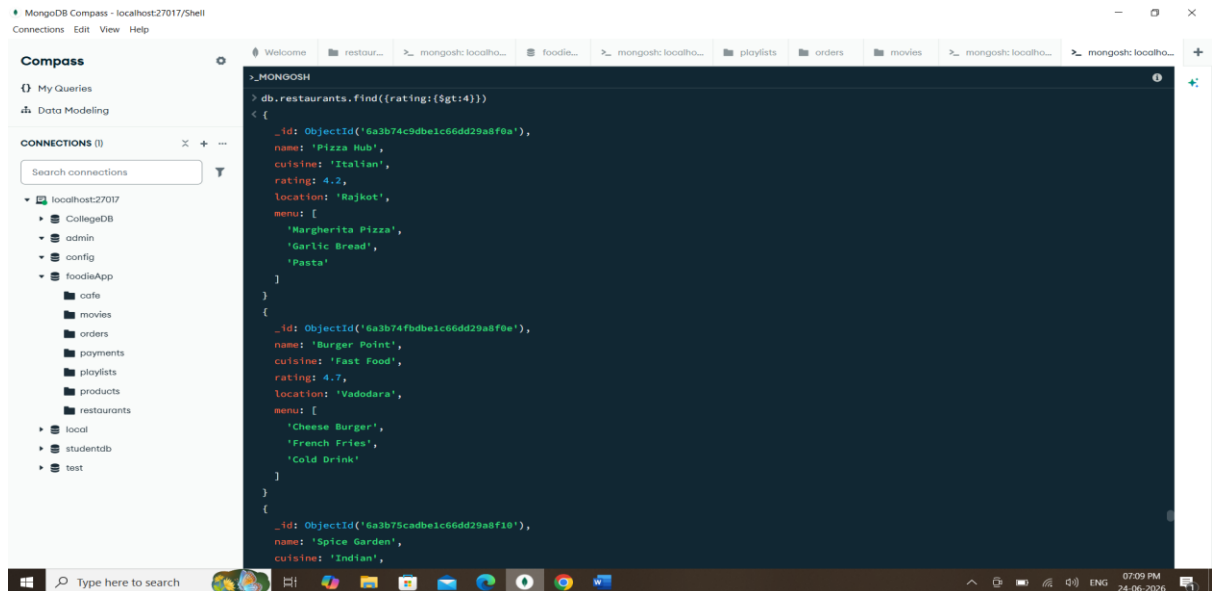
The screenshot shows the MongoDB Compass interface. On the left, the 'CONNECTIONS' panel lists the 'foodieApp' database. The main window displays the MongoDB shell output for the `dropIndexes` command. The output shows a success message: `msg: 'non-_id indexes dropped for collection'`. Below this, the `getIndexes` command is executed, returning an array with one index: `{ v: 2, key: { '_id': 1 }, name: '_id_' }`.

```
> db.restaurants.dropIndexes()
{
  nIndexesWas: 5,
  msg: 'non-_id indexes dropped for collection',
  ok: 1
}

> db.restaurants.getIndexes()
< [ { v: 2, key: { '_id': 1 }, name: '_id_' } ]
foodieApp>
```

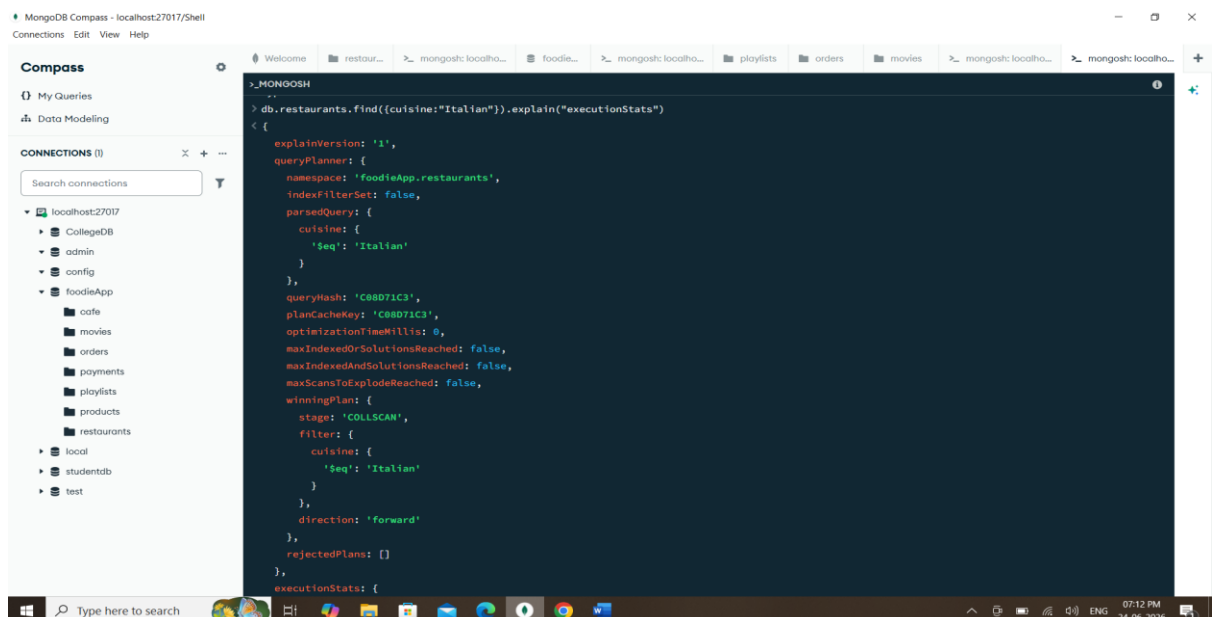
SESSION 9 – Indexing & Performance

1. **Create a MongoDB collection named 'restaurants' and insert at least 10 documents with fields: name, cuisine, location, and rating. Then, run a find query to retrieve all restaurants with rating above 4.**

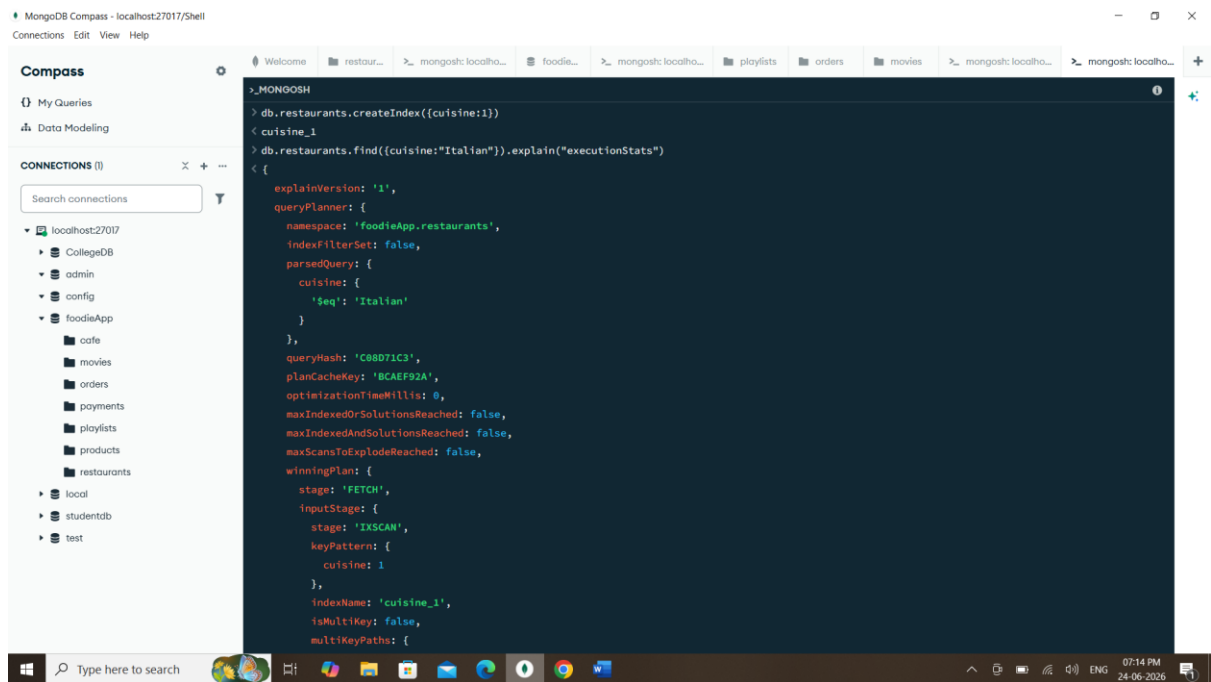


2. Add a single field index on the 'cuisine' field in the 'restaurants' collection. Measure and note the time taken to run a find query filtering by cuisine before and after adding the index. Hint: Use `db.restaurants.find({cuisine: 'Italian'})` and `db.restaurants.createIndex({cuisine: 1})`. Use the 'executionStats' from `explain()` to compare performance.

- **Before Index:**
`db.restaurants.find({cuisine:"Italian"}).explain("executionStats")`



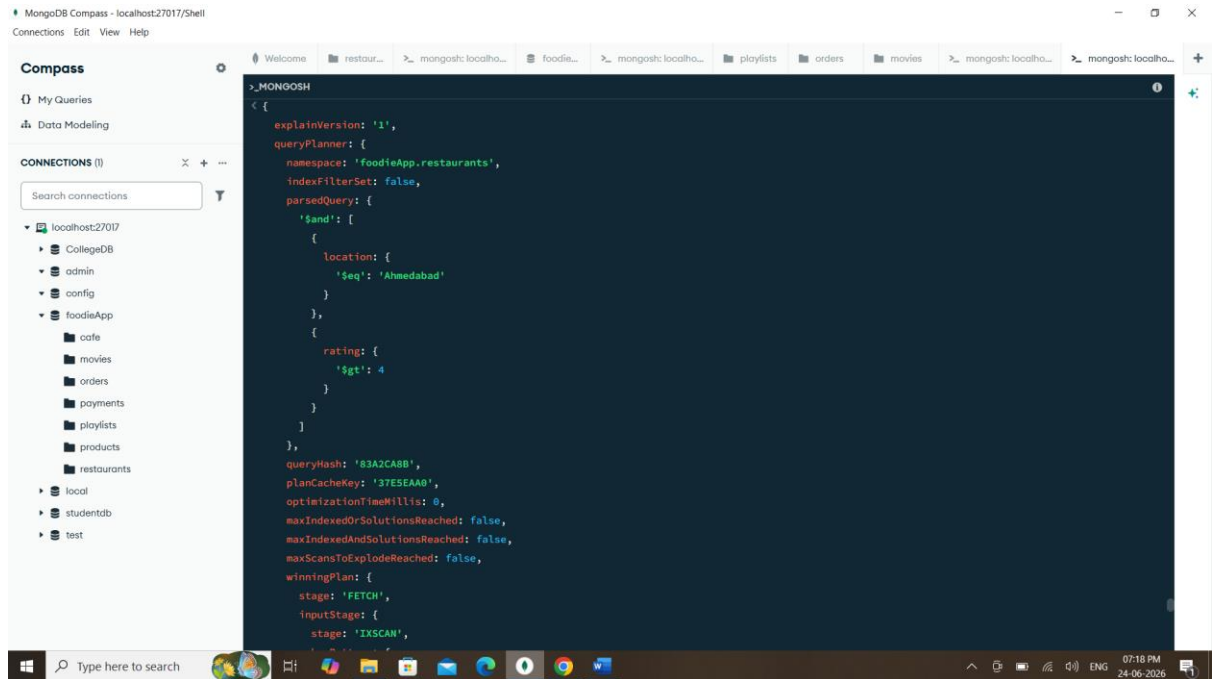
- **Create Index:**
`db.restaurants.createIndex({cuisine:1})`
- **After Index:**
`db.restaurants.find({cuisine:"Italian"}).explain("executionStats")`
- **Observation:**
 - Before index → COLLSCAN
 - After index → IXSCAN
 - executionTimeMillis decreases.



3. Create a compound index on the 'location' and 'rating' fields in the 'restaurants' collection. Then, write and run a query to find all restaurants in 'Ahmedabad' with a rating above 4, and observe the query plan using explain().

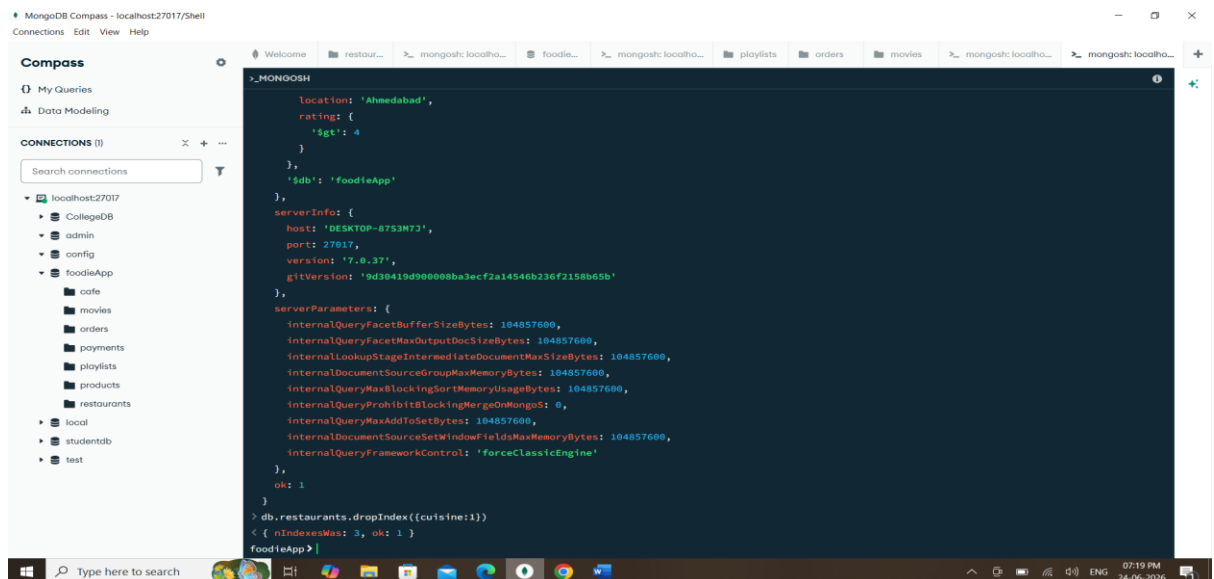
- **Compound Index on Location & Rating**
`db.restaurants.createIndex({location:1,rating:1})`
`db.restaurants.find({`
`location:"Ahmedabad",`
`rating:{>4}`
`}).explain("executionStats")`

- **Observation:**
 - Query planner uses compound index.
 - Search becomes faster.



4. Remove the index on 'cuisine' and observe how the query performance changes for cuisine-based searches. Briefly describe your findings in 2-3 sentences.

- **Remove Cuisine Index**
`db.restaurants.dropIndex({cuisine:1})`
- **Findings:**
 - After removing the cuisine index, MongoDB performs a collection scan (COLLSCAN).
 - Query execution time increases.
 - Cuisine-based searches become slower than indexed searches.



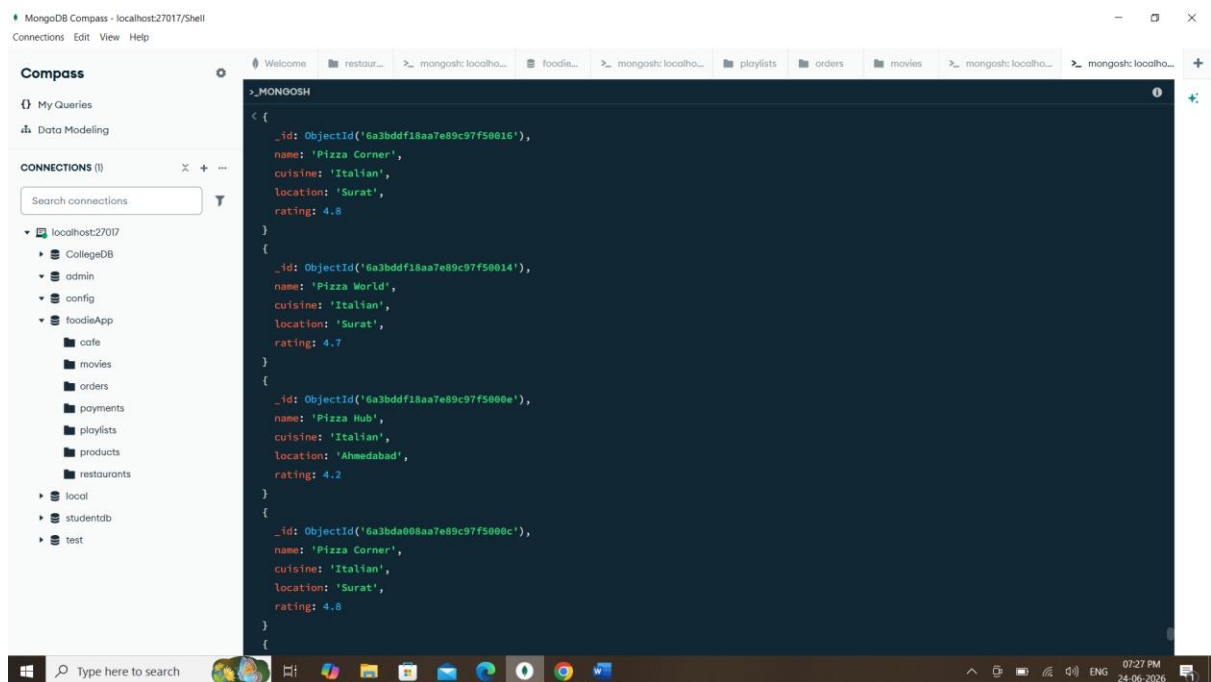
5. Create a text index on the 'name' field of the 'restaurants' collection. Then, write a query to search for restaurants whose name contains the word 'Pizza', and display the results.

- **Create Text Index:**

```
db.restaurants.createIndex({name:"text"})
```

- **Search Query:**

```
db.restaurants.find({
  $text:{$search:"Pizza"}
})
```



SESSION 10 – MangoDB Users & Roles

1. Connect to your MongoDB server using the mongo shell and create a new database called `insta_clone`.

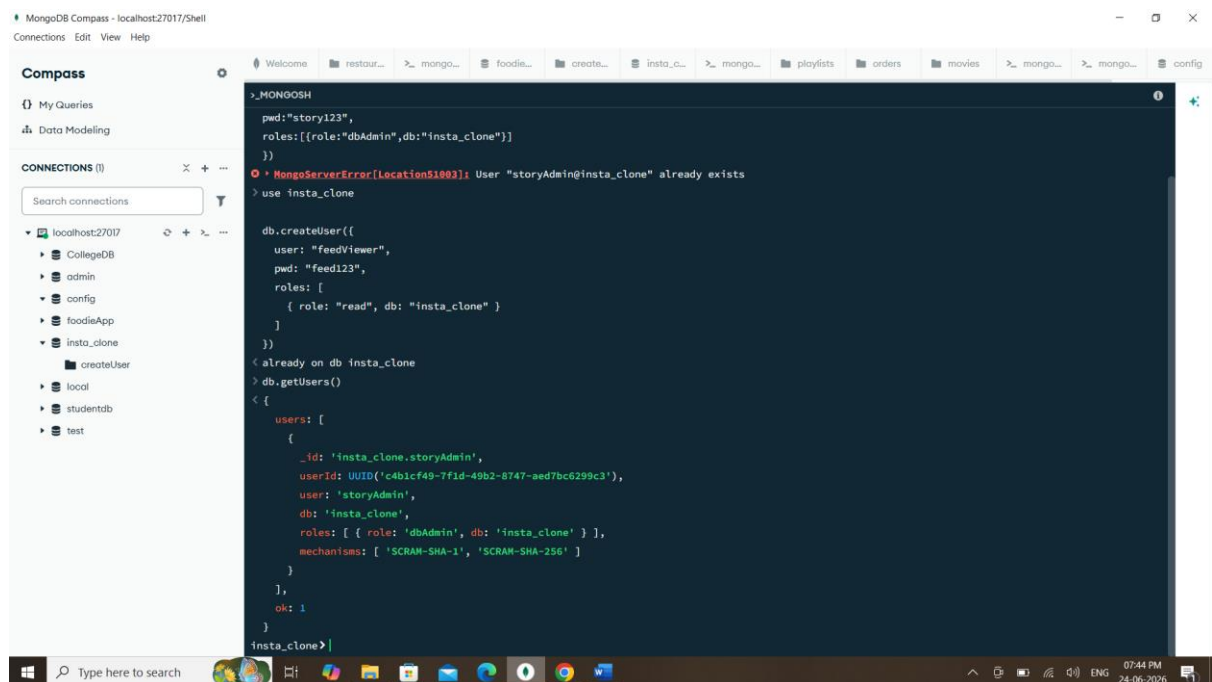
- **Create Database:**

use `insta_clone`

2. Inside the `insta_clone` database, use the `createUser` command to add a user named 'storyAdmin' with the 'dbAdmin' role for managing Instagram Stories data.

- **storyAdmin User Create:**

```
db.createUser({
  user: "storyAdmin",
  pwd: "story123",
  roles: [
    { role: "dbAdmin", db: "insta_clone" }
  ]
})
```



3. Create a read-only user called 'feedViewer' in the insta_clone database who can only view (not modify) posts and comments collections.

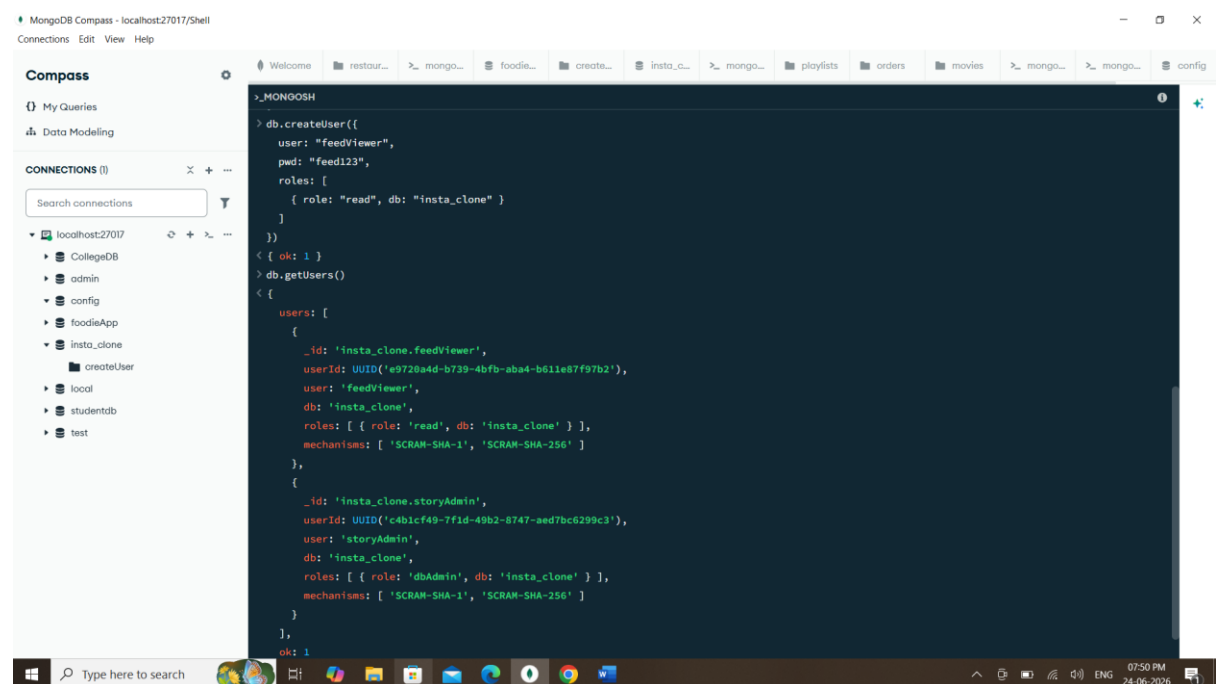
Hint: Use the 'read' role when creating the user.

- **feedViewer User Create:**

```
db.createUser({
  user: "feedViewer",
  pwd: "feed123",
  roles: [
    { role: "read", db: "insta_clone" }
  ]
})
```

- **Users Check:**

```
db.getUsers()
```

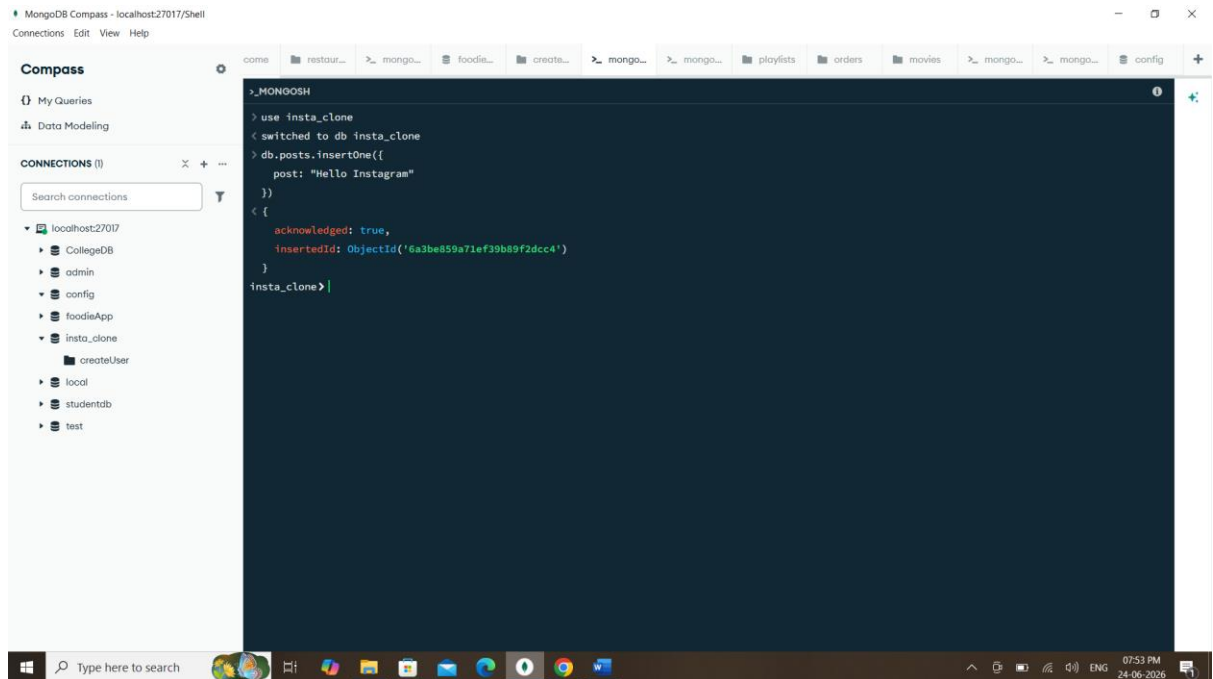


4. **Simulate authentication: Log in as 'feedViewer' and try to insert a new document into the posts collection. Note the error message and explain why it occurs.**

Hint: Use the mongo shell with -u and -p flags.

- **Insert data:**

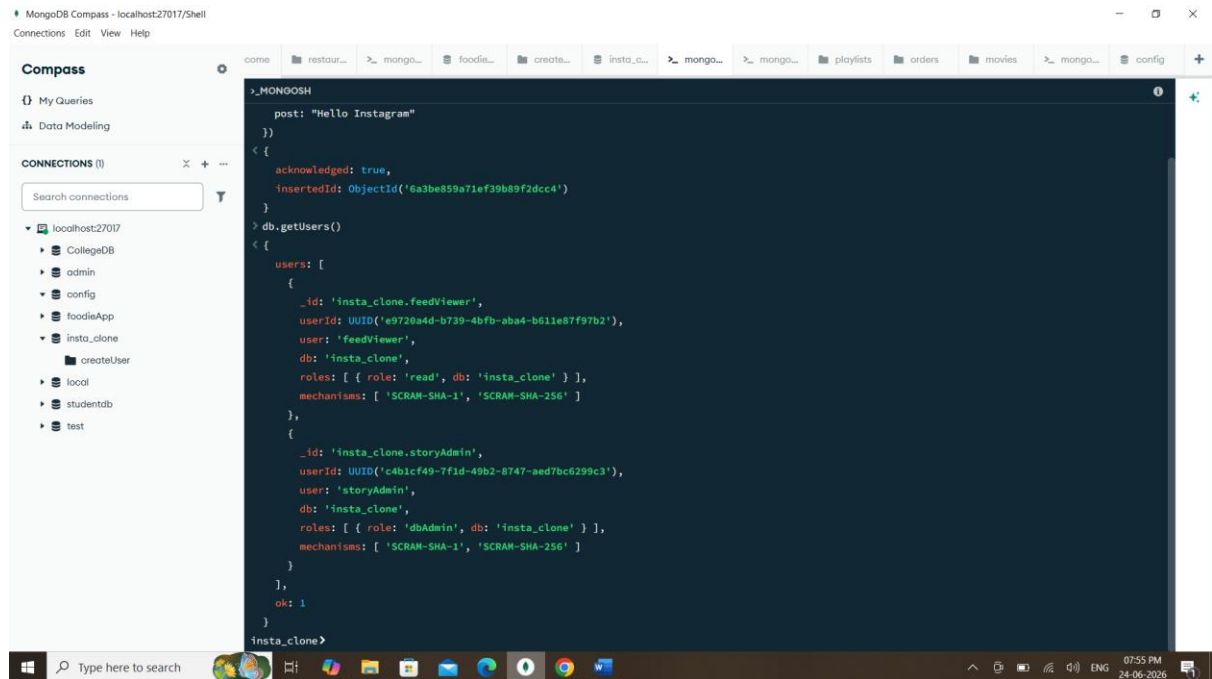
```
db.posts.insertOne({  
  post: "Hello Instagram"  
})
```



5. Use the `db.getUsers()` command to list all users in the `insta_clone` database and describe the roles assigned to each. Then, briefly explain in your own words the difference between 'read' and 'dbAdmin' roles.

- **Query:**

`db.getUsers()`



The screenshot shows the MongoDB Compass interface. On the left, the 'CONNECTIONS' panel lists the 'insta_clone' database. The main editor shows the following JSON output from the `db.getUsers()` command:

```
> db.getUsers()
{
  users: [
    {
      _id: 'insta_clone.feedViewer',
      userId: UUID('e9720a4d-b739-4bfb-aba4-b611e87f97b2'),
      user: 'feedViewer',
      db: 'insta_clone',
      roles: [ { role: 'read', db: 'insta_clone' } ],
      mechanisms: [ 'SCRAM-SHA-1', 'SCRAM-SHA-256' ]
    },
    {
      _id: 'insta_clone.storyAdmin',
      userId: UUID('c4b1cf49-7f1d-49b2-8747-aed7bc6299c3'),
      user: 'storyAdmin',
      db: 'insta_clone',
      roles: [ { role: 'dbAdmin', db: 'insta_clone' } ],
      mechanisms: [ 'SCRAM-SHA-1', 'SCRAM-SHA-256' ]
    }
  ],
  ok: 1
}
```

- **Users Found:**

- storyAdmin → dbAdmin role
- feedViewer → read role

- **Difference (Roles Explanation):**

- read → Only view data
- dbAdmin → Manage database objects and administration tasks

SESSION 11 – MongoDB Users & Roles

1. Start your local MongoDB server and use the mongo shell to create a new database called instaAppData.

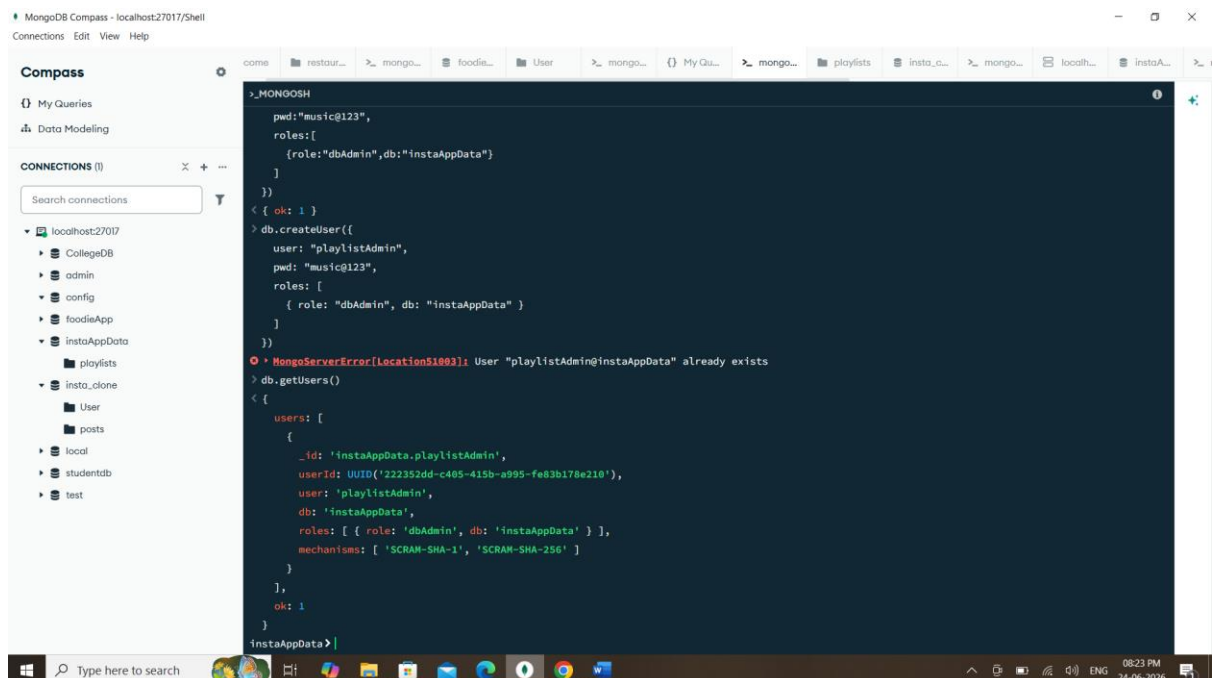
- **Create Database:**
use instaAppData

2. Inside instaAppData, create a new user named 'playlistAdmin' with password 'music@123' and assign the 'dbAdmin' role — this user should be able to manage playlists and collections, but not read or write actual playlist data.

- **Create User playlistAdmin (dbAdmin Role)**

```
db.createUser({
  user:"playlistAdmin",
  pwd:"music@123",
  roles:[
    {role:"dbAdmin",db:"instaAppData"}
  ]
})
```

- **View query: db.getUsers()**

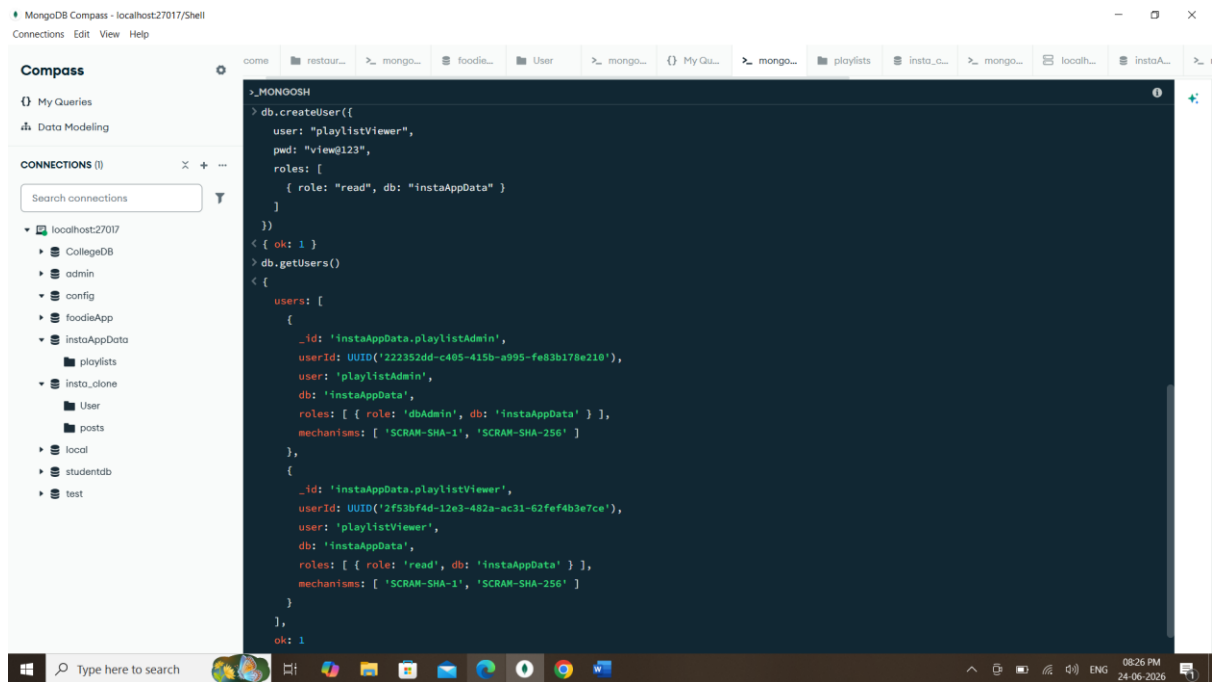


3. Create another user in instaAppData named 'playlistViewer' with password 'view@123' and assign only the 'read' role — this user should only be able to view playlists, not modify or delete them.

- Create User playlistViewer (Read Role)

```
db.createUser({
  user:"playlistViewer",
  pwd:"view@123",
  roles:[
    {role:"read",db:"instaAppData"}
  ]
})
```

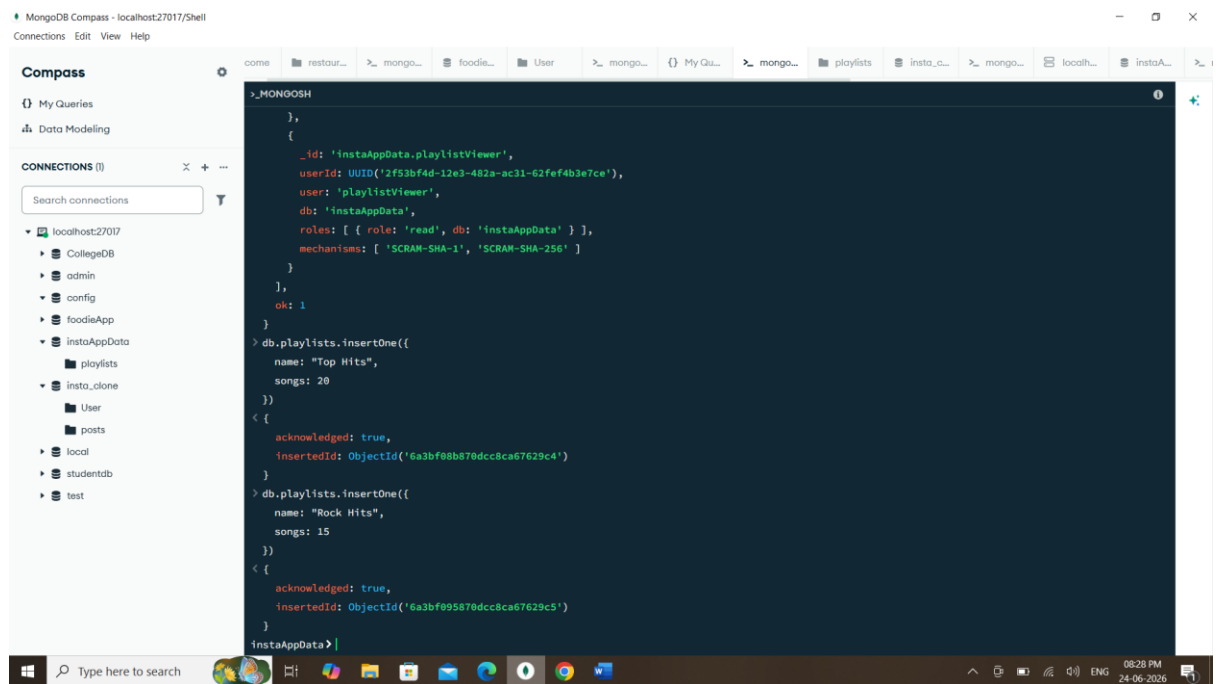
- Check all users: db.getUsers()



4. Test authentication by connecting to instaAppData as 'playlistViewer' and try to insert a new playlist document; note the error message and explain why it occurs. Hint: Use the mongo shell's --username and --password flags to connect as the user.

- Create playlists Collection and Insert Data

```
db.playlists.insertOne({  
  name: "Top Hits",  
  songs: 20  
})
```



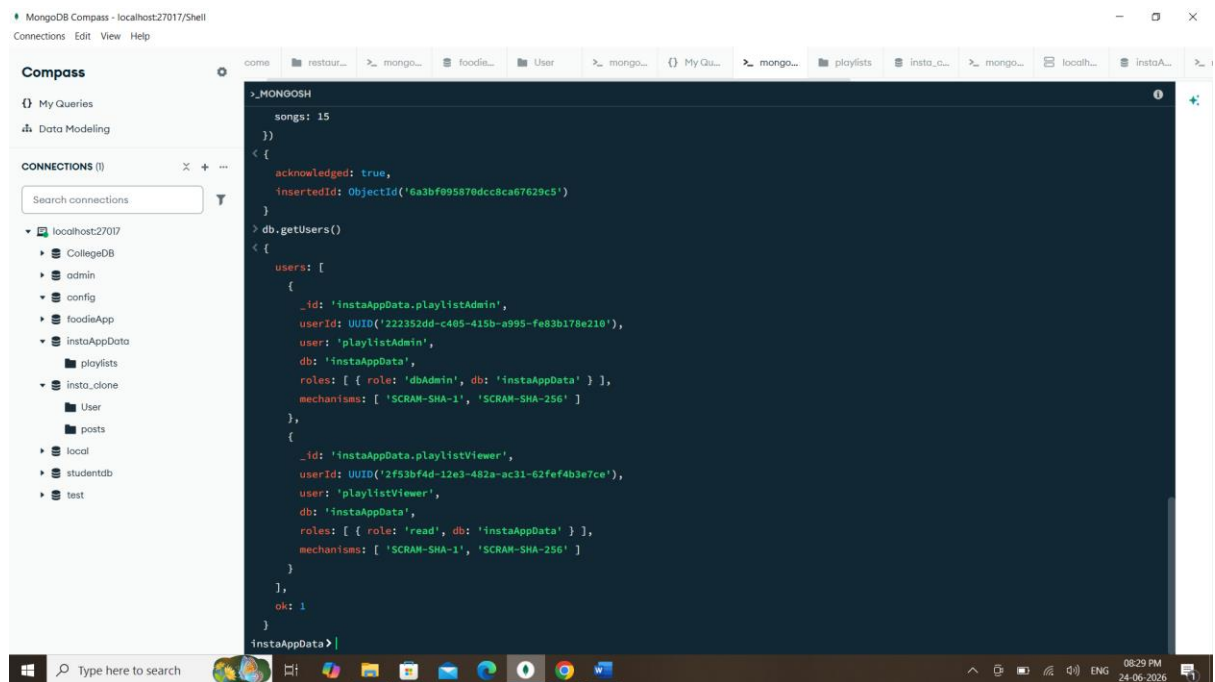
5. Use the mongo shell to list all users and their roles in the instaAppData database, and explain in one line what each role allows the user to do.

- **List All Users**

db.getUsers()

- **Role Explanation:**

- **dbAdmin** → Can manage collections, indexes, and database administration tasks.
- **read** → Can only view data/documents and cannot insert, update, or delete them.

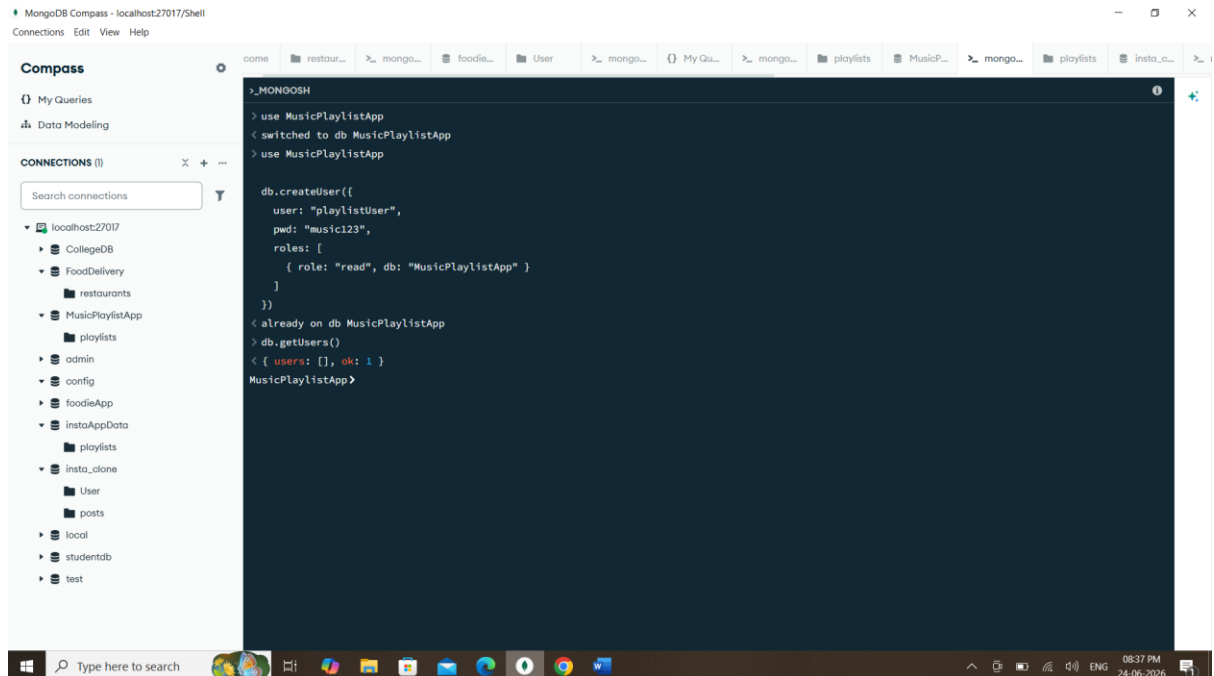


The screenshot shows the MongoDB Compass interface with the mongo shell open. The shell output displays the result of the `db.getUsers()` command, showing two users: `playlistAdmin` and `playlistViewer`, each with their respective roles and mechanisms.

```
> use instaAppData
> db.getUsers()
{
  acknowledged: true,
  insertedIds: {
    '6a3bf095870dcca67629c5': 1
  }
}
> db.getUsers()
{
  users: [
    {
      _id: 'instaAppData.playlistAdmin',
      userId: UUID('222352dd-c405-415b-a995-fe83b178e210'),
      user: 'playlistAdmin',
      db: 'instaAppData',
      roles: [ { role: 'dbAdmin', db: 'instaAppData' } ],
      mechanisms: [ 'SCRAM-SHA-1', 'SCRAM-SHA-256' ]
    },
    {
      _id: 'instaAppData.playlistViewer',
      userId: UUID('2f53bf4d-12e3-482a-ac31-62fef4b3e7ce'),
      user: 'playlistViewer',
      db: 'instaAppData',
      roles: [ { role: 'read', db: 'instaAppData' } ],
      mechanisms: [ 'SCRAM-SHA-1', 'SCRAM-SHA-256' ]
    }
  ],
  ok: 1
}
```

SESSION 11 – MongoDB Users & Roles

1. Open your MongoDB shell and create a new user for a 'Music Playlist App' database named **playlistUser** with password 'music123', granting only read access to the database.

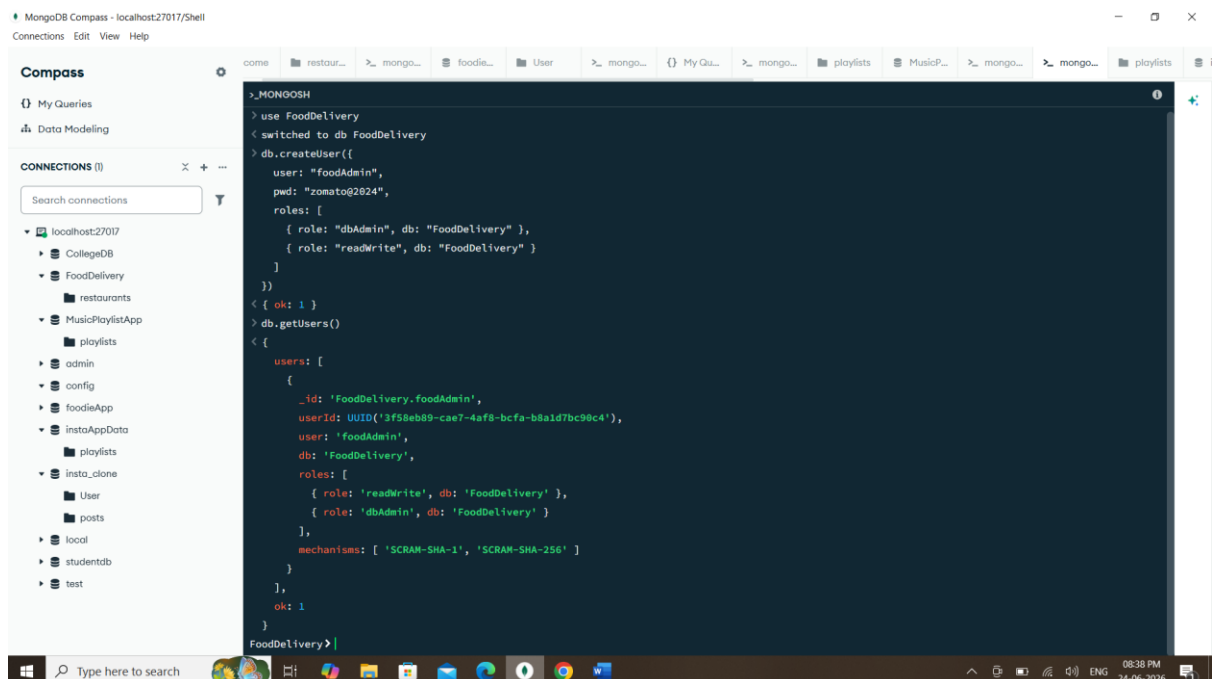


The screenshot shows the MongoDB Compass interface. The left sidebar displays the 'CONNECTIONS' list with various databases. The main panel shows the MongoDB shell with the following commands and output:

```
> use MusicPlaylistApp
switched to db MusicPlaylistApp
> use MusicPlaylistApp

db.createUser({
  user: "playlistUser",
  pwd: "music123",
  roles: [
    { role: "read", db: "MusicPlaylistApp" }
  ]
})
already on db MusicPlaylistApp
> db.getUsers()
{ users: [], ok: 1 }
MusicPlaylistApp>
```

2. In the MongoDB shell, create an admin user for a 'Food Delivery' database named **foodAdmin** with password 'zomato@2024', assigning the 'dbAdmin' and 'readWrite' roles.



The screenshot shows the MongoDB Compass interface. The left sidebar displays the 'CONNECTIONS' list with various databases. The main panel shows the MongoDB shell with the following commands and output:

```
> use FoodDelivery
switched to db FoodDelivery
> db.createUser({
  user: "foodAdmin",
  pwd: "zomato@2024",
  roles: [
    { role: "dbAdmin", db: "FoodDelivery" },
    { role: "readWrite", db: "FoodDelivery" }
  ]
})
{ ok: 1 }
> db.getUsers()
{
  users: [
    {
      _id: 'FoodDelivery.foodAdmin',
      userId: UUID('3f58eb89-cae7-4af8-bcfa-b8ald7bc98c4'),
      user: 'foodAdmin',
      db: 'FoodDelivery',
      roles: [
        { role: 'readWrite', db: 'FoodDelivery' },
        { role: 'dbAdmin', db: 'FoodDelivery' }
      ],
      mechanisms: [ 'SCRAM-SHA-1', 'SCRAM-SHA-256' ]
    }
  ],
  ok: 1
}
FoodDelivery>
```


3. **Authenticate as playlistUser in the 'Music Playlist App' database and try to insert a new playlist document. Note the error message and explain why it occurs.**

Hint: playlistUser only has read access.

****Authenticate as playlistUser and Try to Insert a Playlist****

- **Login Command:**

```
```bash
mongosh -u playlistUser -p music123 --authenticationDatabase MusicPlaylistApp
```
```

- **Insert Query:**

```
```javascript
db.playlists.insertOne({
 name: "My Playlist",
 songs: 10
})
```
```

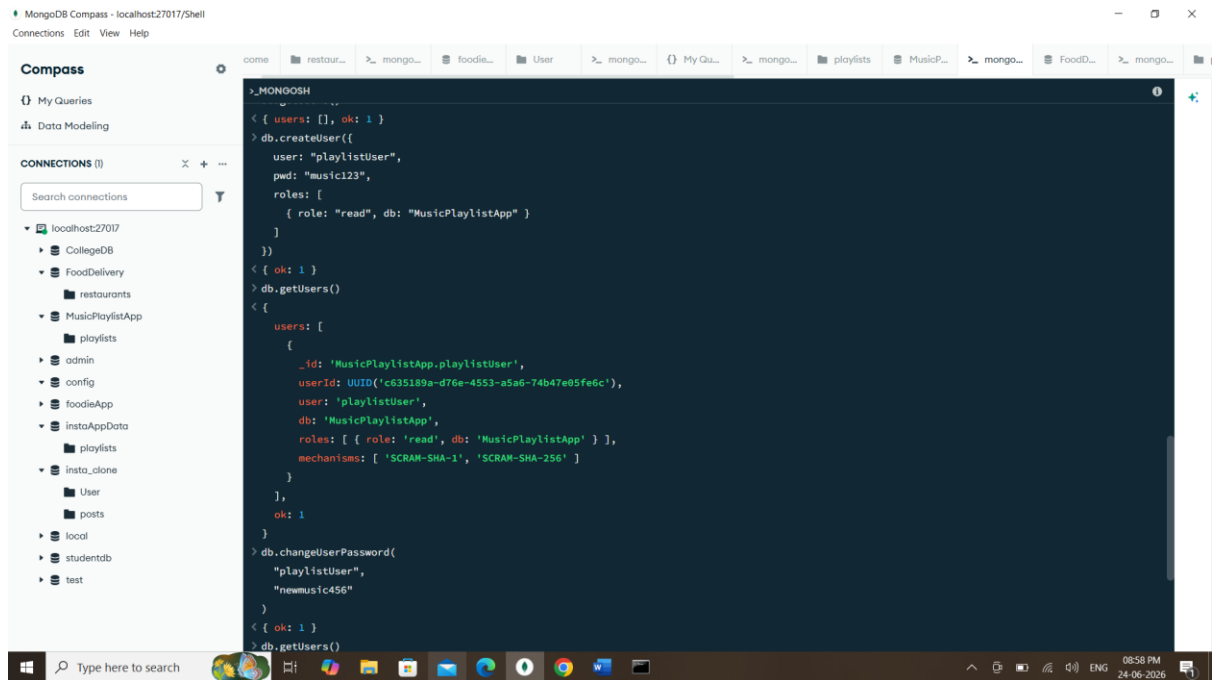
- **Error Message:**

```
```text
MongoServerError: not authorized on MusicPlaylistApp to execute command insert
```
```

- **Explanation:**

The error occurs because ****playlistUser**** has only the ****read**** role on the MusicPlaylistApp database. A user with the read role can view documents but cannot insert, update, or delete data. Therefore, MongoDB denies the insert operation.

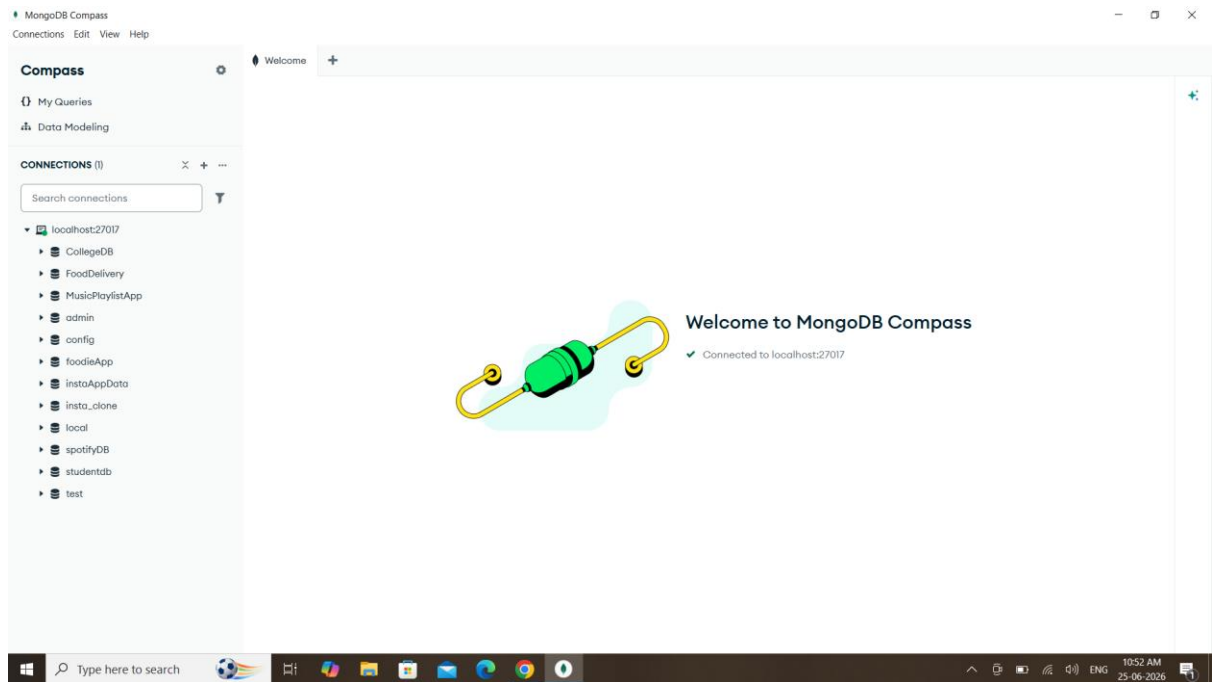
5. Change the password for playlistUser to 'newmusic456' using the MongoDB shell, and verify that the user can still read but not write data.



SESSION 13 – MangoDB Tools

1. Install MongoDB Compass on your system, connect to your local MongoDB server, and create a new database named spotifyDB with a collection called playlists.

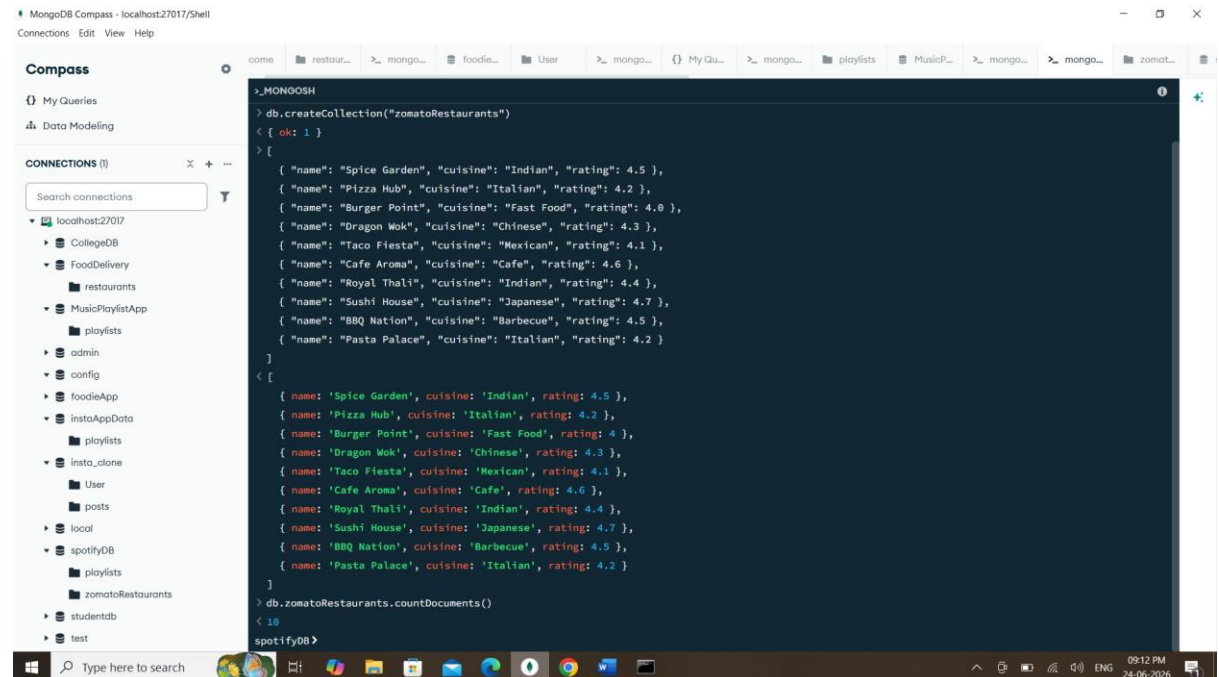
- **Create Database spotifyDB and Collection playlists**
- **MongoDB Compass** → Create Database
- **Database Name:** spotifyDB
- **Collection Name:** playlists



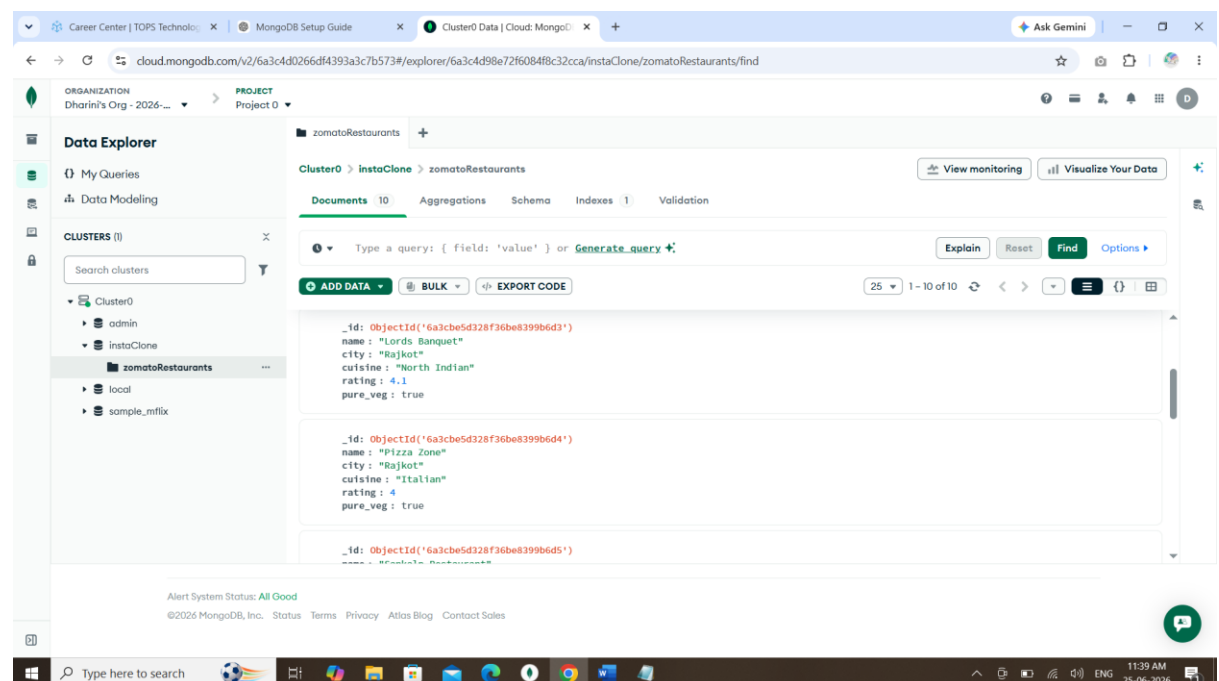
2. Using MongoDB Compass, import a JSON file containing at least 10 sample restaurant documents (name, cuisine, rating) into a new collection named `zomatoRestaurants` in your `spotifyDB` database.

Hint: You can find sample restaurant JSON data online or create your own small dataset.

- Import JSON Data into `zomatoRestaurants`



3. Sign up for a free MongoDB Atlas account, create a new cluster, and deploy a database named `instaClone` on Atlas.



4. Export the `zomatoRestaurants` collection from your local MongoDB using the `mongoexport` tool and then import it into your `instaClone` database on MongoDB Atlas using the `mongoimport` tool.

Hint: Use the connection string provided by MongoDB Atlas for the import step.

The screenshot shows the MongoDB Atlas Data Explorer interface. The left sidebar displays the 'Data Explorer' section with a tree view of clusters. The main panel shows the 'zomatoRestaurants' collection under the 'Cluster0' cluster. The 'Documents' tab is active, displaying a table of 10 documents. The table has columns for '_id', 'name', 'city', 'cuisine', and 'rating'. The documents are listed with their ObjectIds and corresponding restaurant details.

| _id | name | city | cuisine | rating |
|--------------------------------|-----------------------|----------|--------------------|--------|
| ObjectId('6a3cbe5d328f36b...') | "The Grand Thakar" | "Rajkot" | "Gujarati Thali" | 4.5 |
| ObjectId('6a3cbe5d328f36b...') | "Imperial Palace" | "Rajkot" | "Multi-Cuisine" | 4.3 |
| ObjectId('6a3cbe5d328f36b...') | "Lords Banquet" | "Rajkot" | "North Indian" | 4.1 |
| ObjectId('6a3cbe5d328f36b...') | "Pizza Zone" | "Rajkot" | "Italian" | 4 |
| ObjectId('6a3cbe5d328f36b...') | "Sankalp Restaurant" | "Rajkot" | "South Indian" | 4.4 |
| ObjectId('6a3cbe5d328f36b...') | "Subway" | "Rajkot" | "Fast Food" | 4.2 |
| ObjectId('6a3cbe5d328f36b...') | "Red Pepper" | "Rajkot" | "Chinese" | 3.9 |
| ObjectId('6a3cbe5d328f36b...') | "Flavours Restaurant" | "Rajkot" | "Continental" | 4.2 |
| ObjectId('6a3cbe5d328f36b...') | "Big Bite" | "Rajkot" | "Burgers & Shakes" | 3.8 |

5. Connect to your MongoDB Atlas `instaClone` database using MongoDB Compass, and verify that all restaurant documents have been successfully imported by viewing the `zomatoRestaurants` collection and counting the documents.

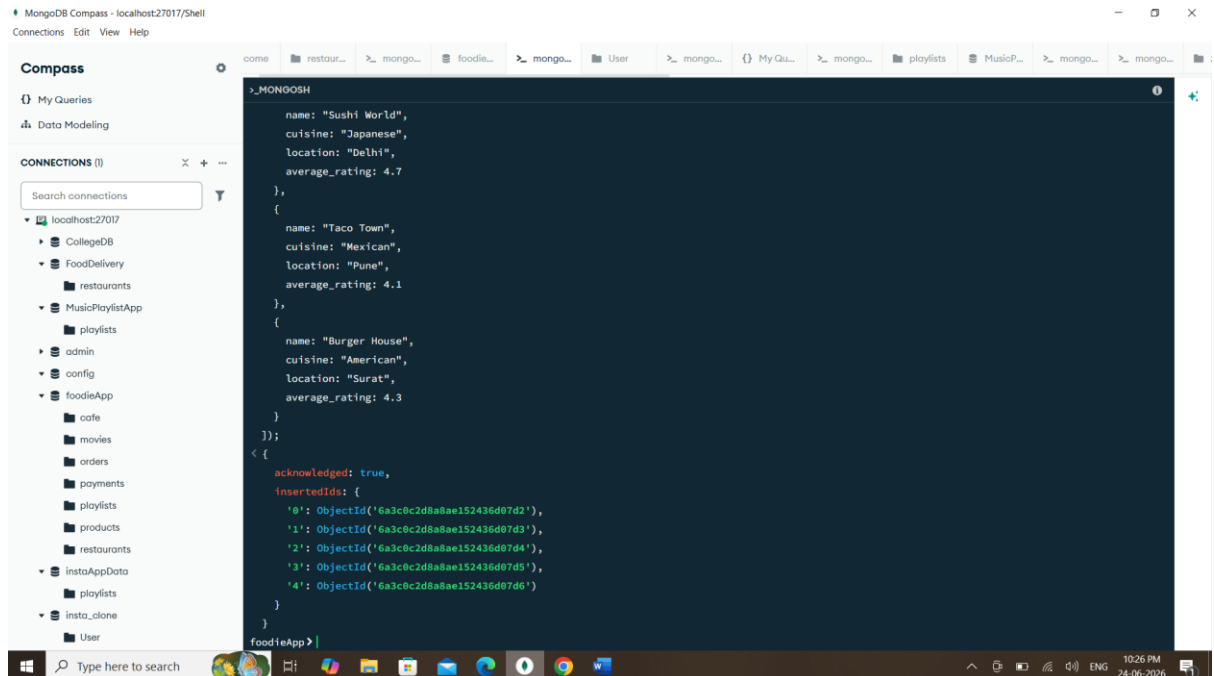
The screenshot shows the MongoDB Atlas Data Explorer interface, similar to the previous one, but with the 'Documents' tab expanded to show individual document details. The first document is selected, showing its full JSON structure. The second document is also visible below it.

```
{
  "_id": "ObjectId('6a3cbe5d328f36be8399b6d1')",
  "name": "The Grand Thakar",
  "city": "Rajkot",
  "cuisine": "Gujarati Thali",
  "rating": 4.5,
  "pure_veg": true
}
```

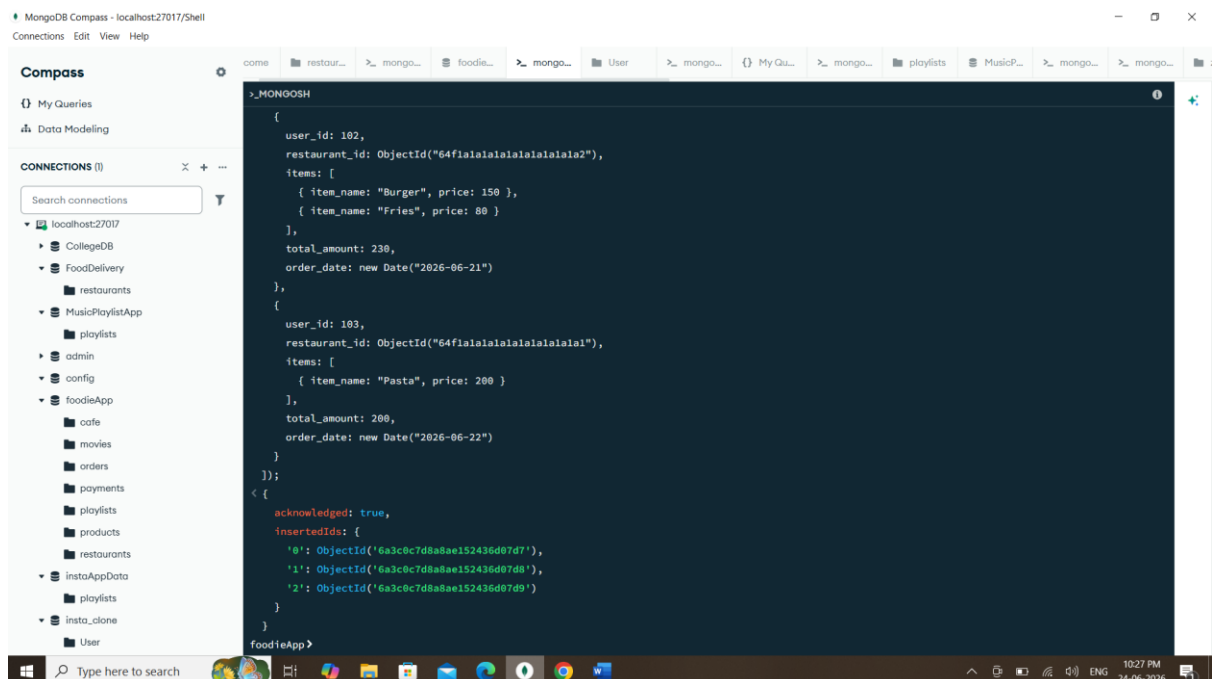
```
{
  "_id": "ObjectId('6a3cbe5d328f36be8399b6d2')",
  "name": "Imperial Palace",
  "city": "Rajkot",
  "cuisine": "Multi-Cuisine",
  "rating": 4.3,
  "pure_veg": false
}
```

SESSION 14 – Mini Project

1. Create a MongoDB collection named 'restaurants' and insert 5 documents, each representing a different restaurant with fields: name, cuisine, location, and average_rating.



2. Design an 'orders' collection for a food delivery app (like Zomato or Swiggy) where each order document includes: user_id, restaurant_id, items (array of objects with item_name and price), total_amount, and order_date. Insert 3 sample orders.

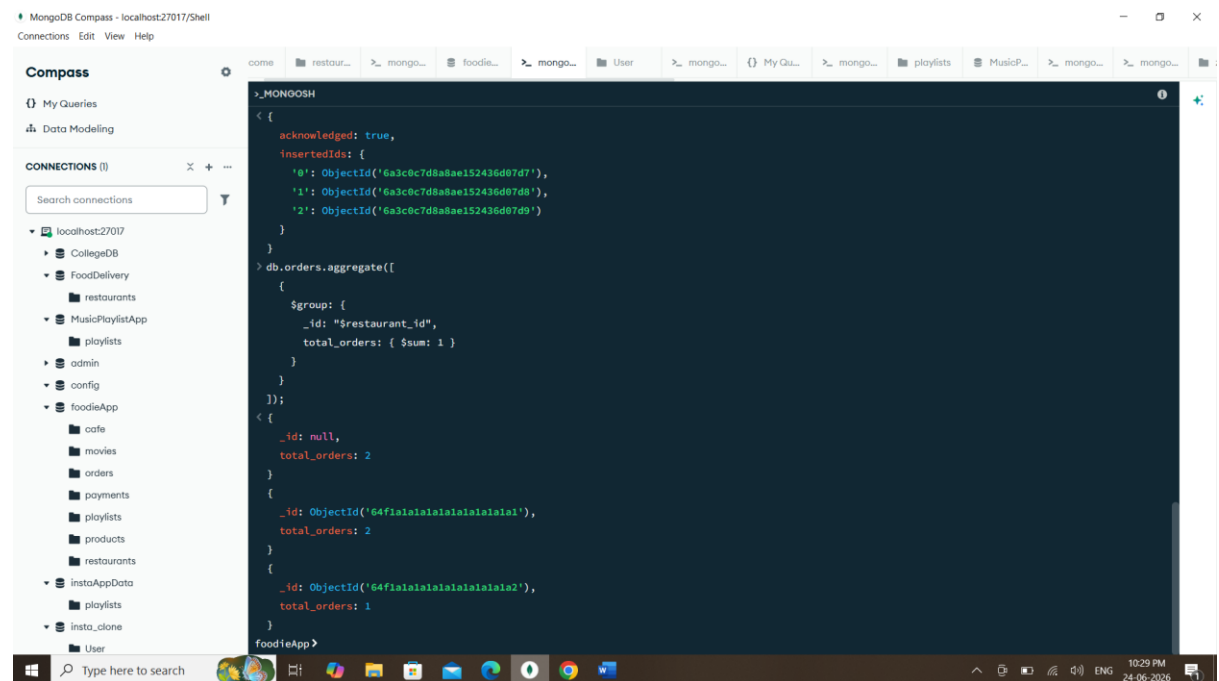


3. Write a MongoDB aggregation query to calculate the total number of orders placed for each restaurant in the 'orders' collection and display the restaurant_id with its corresponding order count.

Hint: Use the \$group stage in the aggregation pipeline.

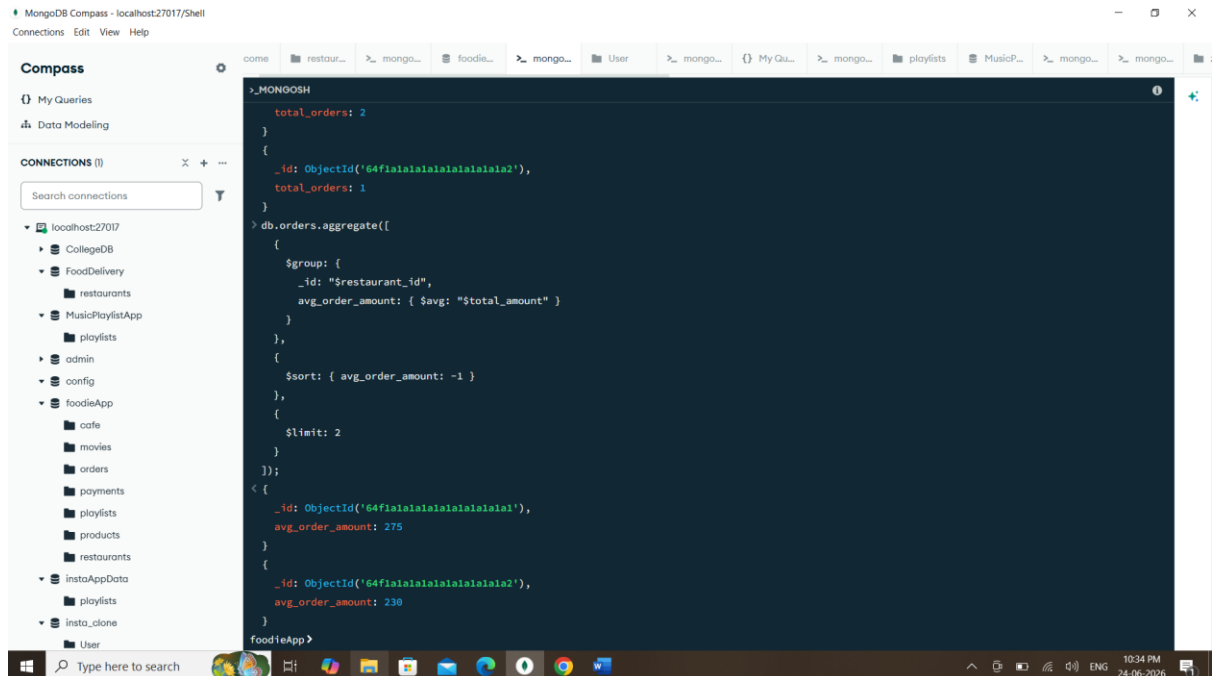
- Query:

```
db.orders.aggregate([{$group: {_id: "$restaurant_id", total_orders: { $sum: 1 } } }]);
```



4. Generate a report using MongoDB aggregation that finds the top 2 restaurants with the highest average order amount from your 'orders' collection.

Hint: Use \$group to calculate average, then \$sort and \$limit.

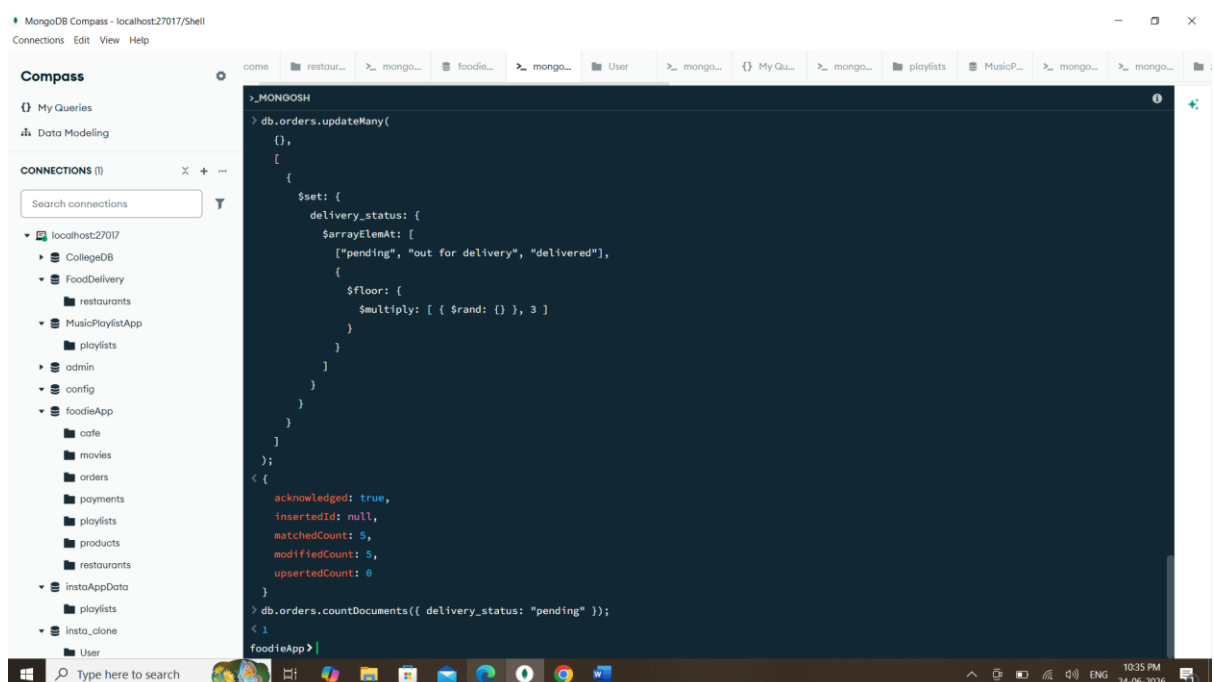


The screenshot shows the MongoDB Compass interface. On the left, the 'Connections' panel shows a connection to 'localhost:27017' with a tree view of databases and collections. The 'foodieApp' database is expanded, showing collections like 'cafe', 'movies', 'orders', 'payments', 'playlists', 'products', 'restaurants', 'instaAppData', 'insta_clone', and 'User'. The main panel displays a MongoDB aggregation query in the shell:

```
>_MONGOSH
total_orders: 2
}
{
  _id: ObjectId('64f1a1a1a1a1a1a1a2'),
  total_orders: 1
}
> db.orders.aggregate([
  {
    $group: {
      _id: "$restaurant_id",
      avg_order_amount: { $avg: "$total_amount" }
    }
  },
  {
    $sort: { avg_order_amount: -1 }
  },
  {
    $limit: 2
  }
]);
< {
  _id: ObjectId('64f1a1a1a1a1a1a1a1'),
  avg_order_amount: 275
}
{
  _id: ObjectId('64f1a1a1a1a1a1a1a2'),
  avg_order_amount: 230
}
foodieApp >
```

The results show the top 2 restaurants based on their average order amount. The first restaurant has an average order amount of 275, and the second has 230.

5. Refactor your 'orders' collection to include a 'delivery_status' field (values: 'pending', 'out for delivery', 'delivered'). Update all existing orders to randomly assign one of these statuses, then write a query to count how many orders are currently 'pending'.



The screenshot shows the MongoDB Compass interface. The main panel displays a MongoDB updateMany query in the shell:

```
>_MONGOSH
> db.orders.updateMany(
  {},
  [
    {
      $set: {
        delivery_status: {
          $arrayElemAt: [
            ["pending", "out for delivery", "delivered"],
            {
              $floor: {
                $multiply: [ { $rand: {} }, 3 ]
              }
            }
          ]
        }
      }
    }
  ]
);
< {
  acknowledged: true,
  insertedId: null,
  matchedCount: 5,
  modifiedCount: 5,
  upsertedCount: 0
}
> db.orders.countDocuments({ delivery_status: "pending" });
< 1
foodieApp >
```

The update query sets the 'delivery_status' field for all documents in the 'orders' collection to a random value from the array ['pending', 'out for delivery', 'delivered']. The result shows that 5 documents were matched and modified. The subsequent count query shows that there is 1 document with the status 'pending'.